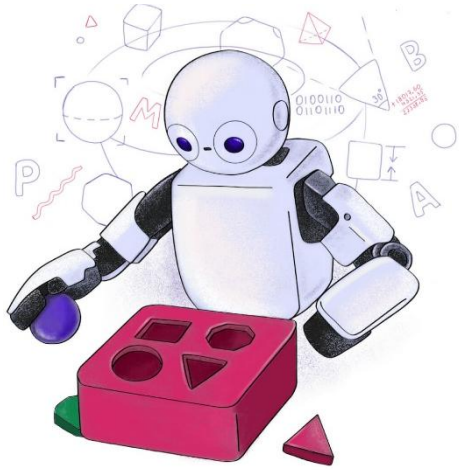
C24 – Inteligência Artificial:
Redes Neurais Convolucionais



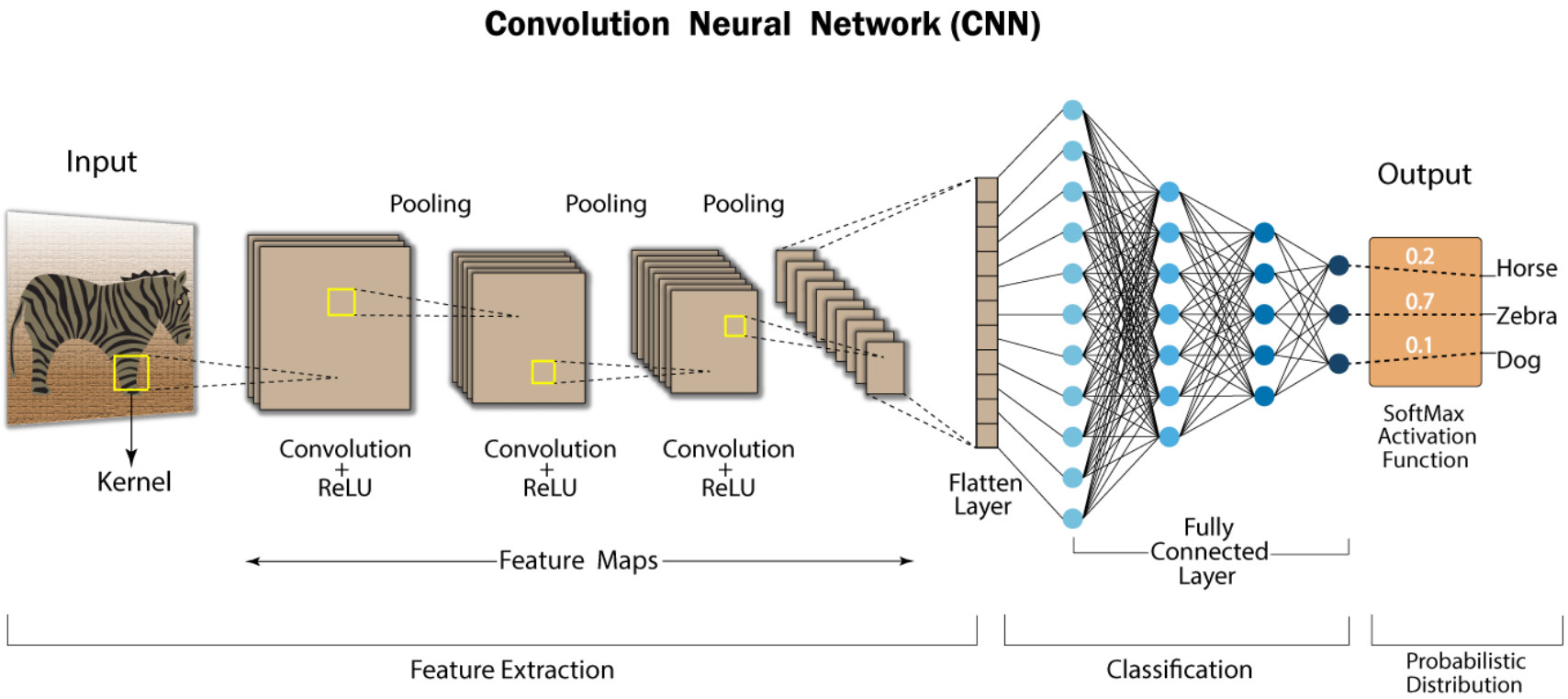
Inatel

Felipe A. P. de Figueiredo
felipe.figueiredo@Inatel.br



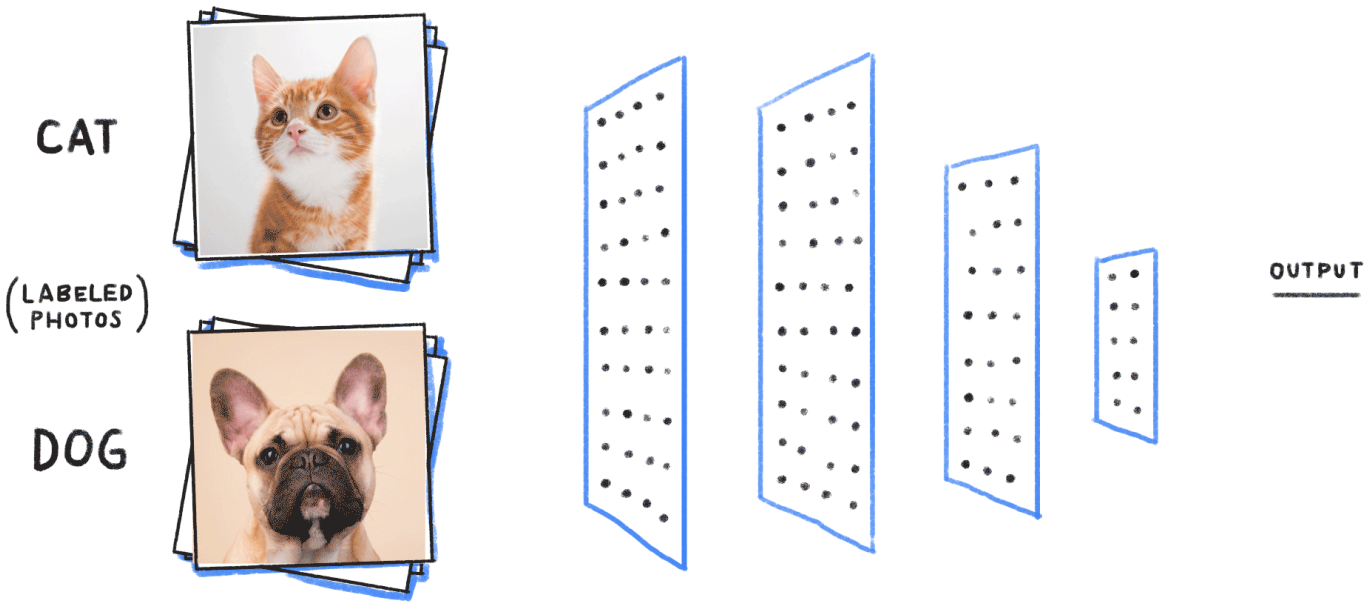
- Até agora, nossas redes neurais continham apenas **camadas densas.**
- Porém, um outro tipo muito importante são as **camadas convolucionais**.
- Essas camadas são a base das *Convolutional Neural Networks* (CNNs).

DNNs versus CNNs



- A **principal diferença** para uma rede densa (DNN) é que ao invés de **aprender relações globais entre os atributos**, uma **CNN aprende relações (padrões) locais,** que podem aparecer em qualquer lugar de uma imagem, **através de filtros de convolução** (ou ***kernels***).

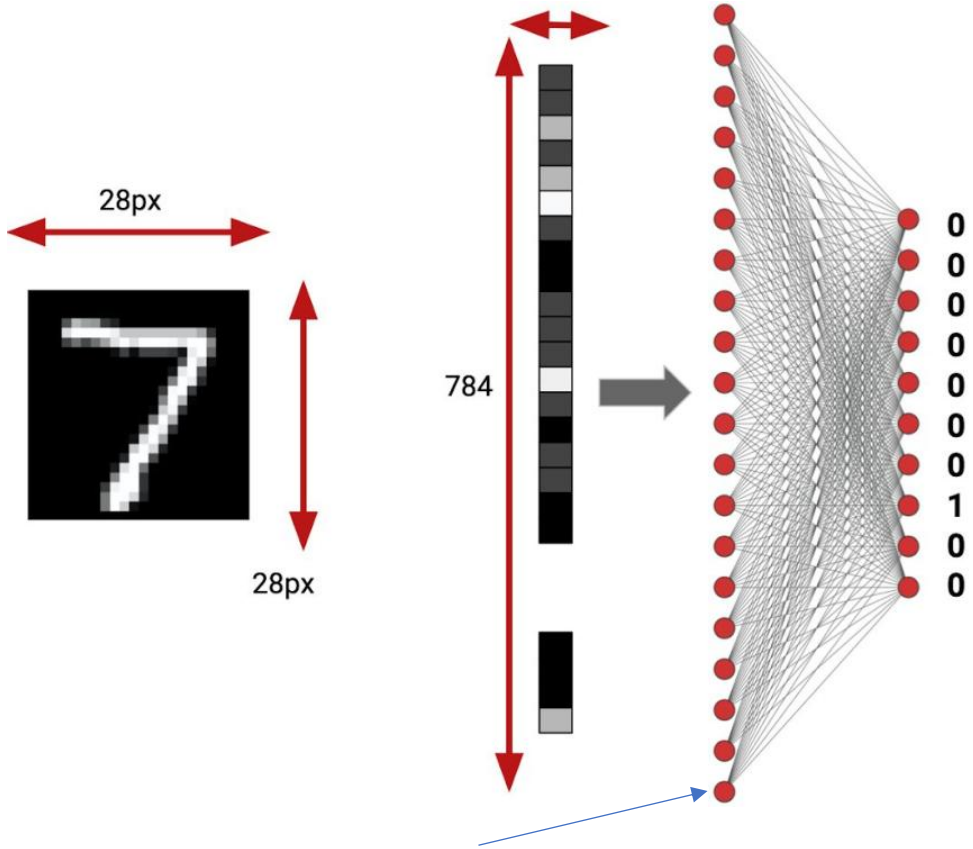
O que vamos ver?



- Em geral, CNNs são usadas em tarefas de **visão computacional**, como, por exemplo
  - classificação de imagens,
  - detecção de objetos,
  - segmentação de objetos,
  - reconhecimento facial e
  - análise médica.

Por que não usamos redes densas (DNNs)?

Dados tabulares



$$g_N = w_0 + x_1 w_1 + x_2 w_2 + \cdots + x_{784} w_{784}$$

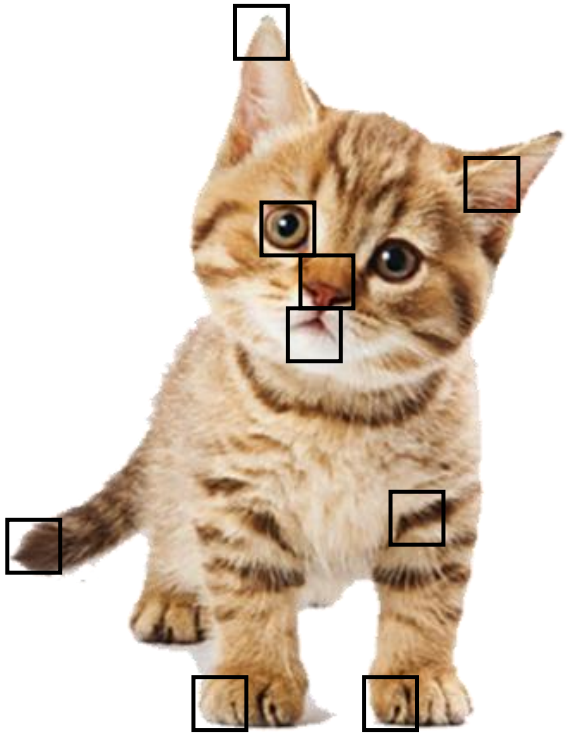
A MLP aprende o que é um 7 **relacionando todos** os 784 *pixels*.

- DNNs são **adequadas para dados tabulares**, onde os **exemplos são representados por linhas e os atributos por colunas**.
- O objetivo de uma MLP é **aprender relações globais entre os atributos**, sem **considerar uma estrutura espacial ou de vizinhança** entre eles.
- DNNs **não possuem**, por construção, **conhecimento de que *pixels* próximos formam uma estrutura espacial (i.e., estão relacionados)**.

Localidade espacial



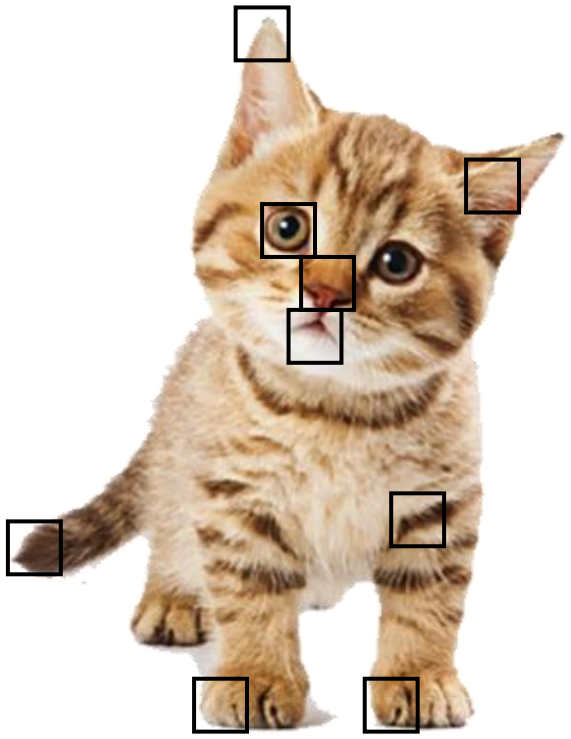
DNN: neurônios recebem todos os *pixels* como entrada



CNN: neurônios recebem (olham) apenas uma parte dos *pixels* da imagem

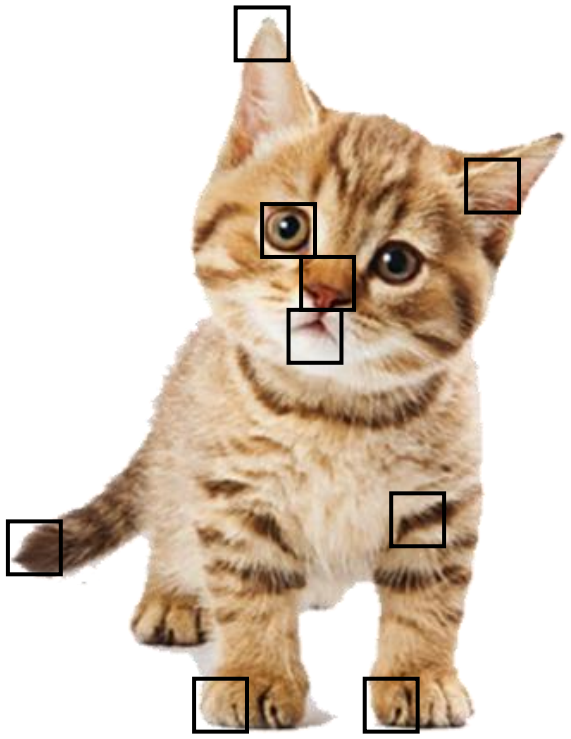
- DNNs **aprendem padrões globais**.
- Elas **não incorporam explicitamente** a **localidade espacial** em dados com topologia de grade (e.g., imagens).
  - **Localidade espacial**: **região** onde **informações próximas** umas das outras **estão relacionadas**.
  - **Topologia em grade**: cada elemento possui uma posição e uma vizinhança espacial bem definida.

Localidade espacial



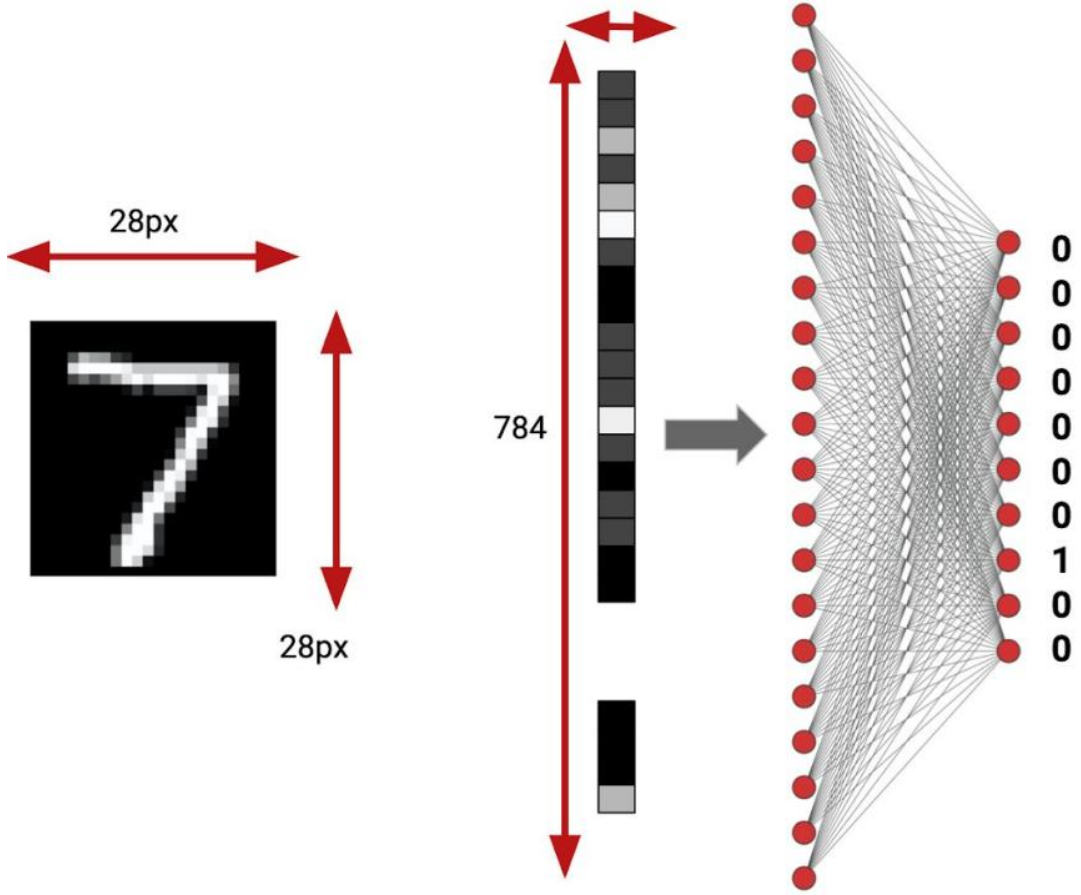
- **Imagens têm uma estrutura espacial** que pode ser explorada por CNNs.
- Os *pixels* possuem **posição no espaço**.
- Em uma imagem, *pixels* **vizinhos** tendem a conter informações **relacionadas**.
- Por exemplo, no caso do gato ao lado, existem **correlações locais** entre os *pixels* dos olhos, nariz, boca, etc.

Localidade espacial



- **Estruturas** (ou **padrões**) importantes como **bordas, contornos, texturas, formas, partes de objetos e objetos inteiros dependem da organização espacial**.
- Portanto, a exploração da **localidade espacial** é **benéfica** para tarefas de **reconhecimento de padrões**.
- As CNNs exploram a localidade espacial usando filtros que percorrem a imagem em busca de padrões.

DNNs e imagens

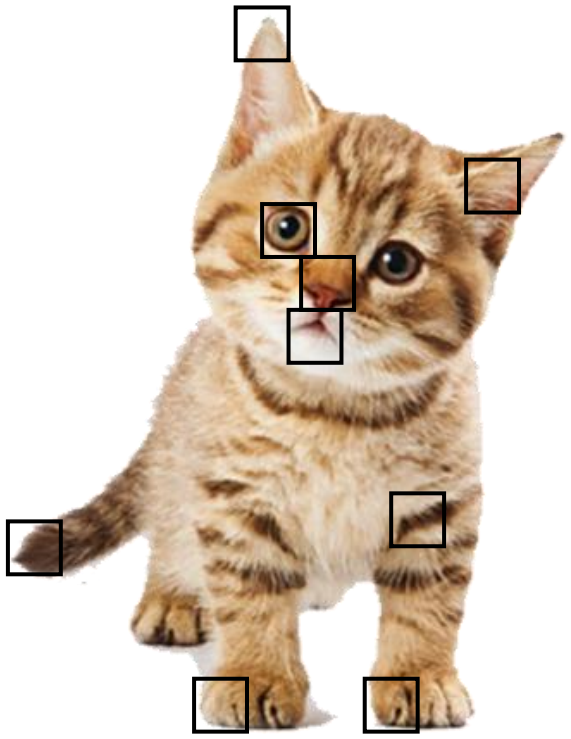


- DNNs são **computacionalmente ineficientes** para processamento de imagens.
- Redes densas **conectam cada *pixel* a todos os nós de uma camada**, gerando um número enorme de pesos, o que resulta em uma **rede muito grande**.
- Exemplo:
  - Uma imagem de 28 x 28 *pixels* gera uma entrada com 784 atributos.
  - Se a primeira camada tiver 100 neurônios serão 784 x 100 = 78400 pesos.
  - E isso apenas na primeira camada.

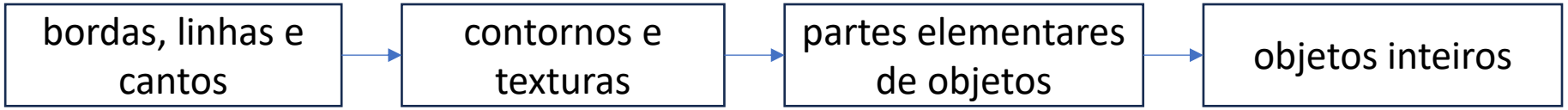
DNNs e imagens

- Em uma DNN, o **número de pesos cresce com a resolução da imagem**.
- Problemas causados por um modelo grande
  - alto custo computacional
    - treinamento mais lento
    - maior consumo de memória
  - maior risco de *overfitting*
- Em geral, para ter desempenho similar, DNNs precisam ser maiores do que CNNs, pois precisam de mais nós para apreender o mesmo padrão em diferentes posições.
- Além disso, por **desconsiderarem a estrutura espacial** dos *pixels*, seriam necessárias **redes e *datasets* extremamente grandes** para funcionarem bem com **problemas de detecção e segmentação de objetos**, os quais se **baseiam na localização dos objetos**.

DNNs e imagens

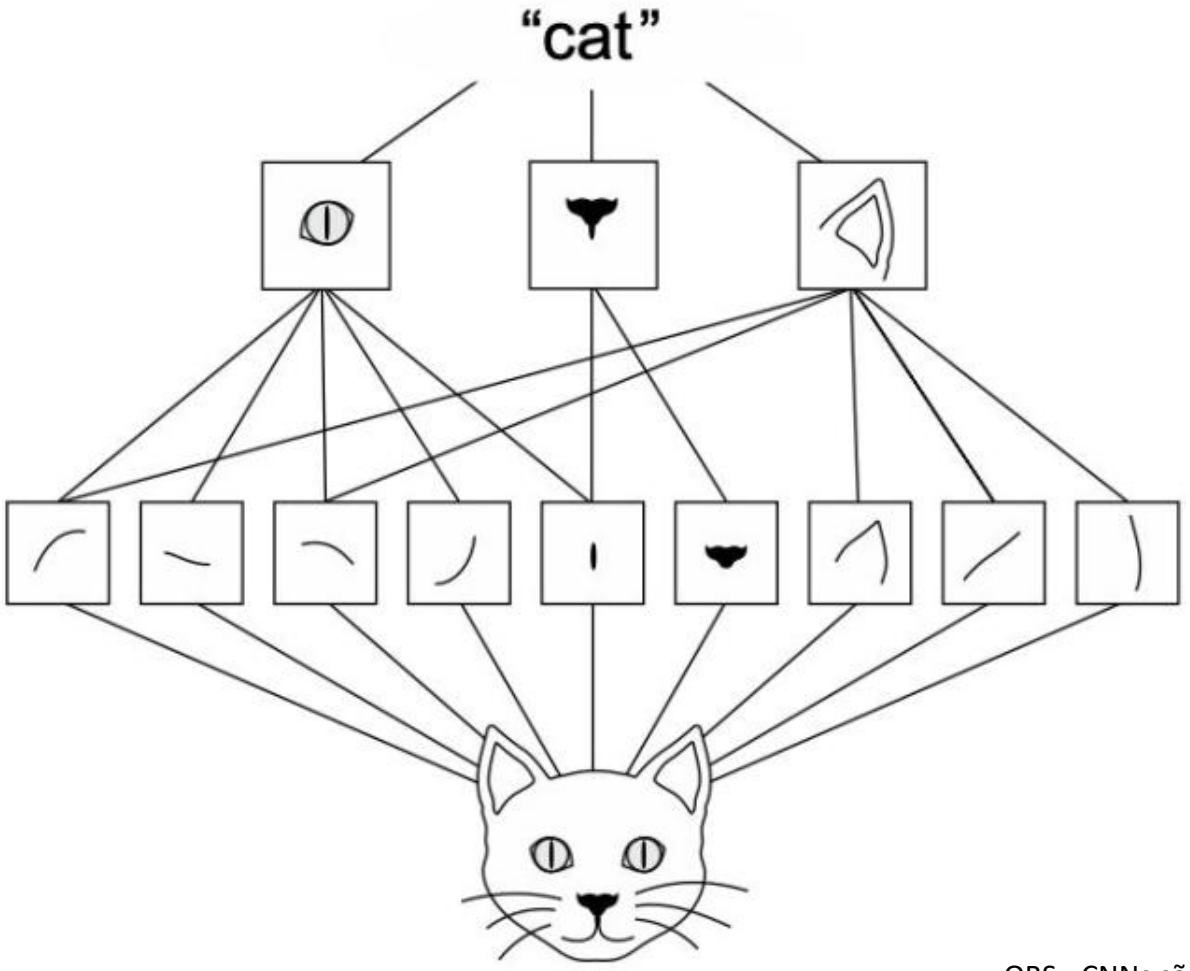


- Portanto, a **conectividade total dos nós em uma DNN é um desperdício** para tarefas de **processamento de imagens**, as quais são
  - **dominadas por padrões espacialmente locais**
  - **e contêm hierarquias.**
    - Características simples se unem para criar características mais complexas.
    - Por exemplo, bordas, linhas e cantos formam contornos e texturas, que formam partes elementares de objetos e, que por sua vez, formam objetos completos.



Por que usar CNNs?

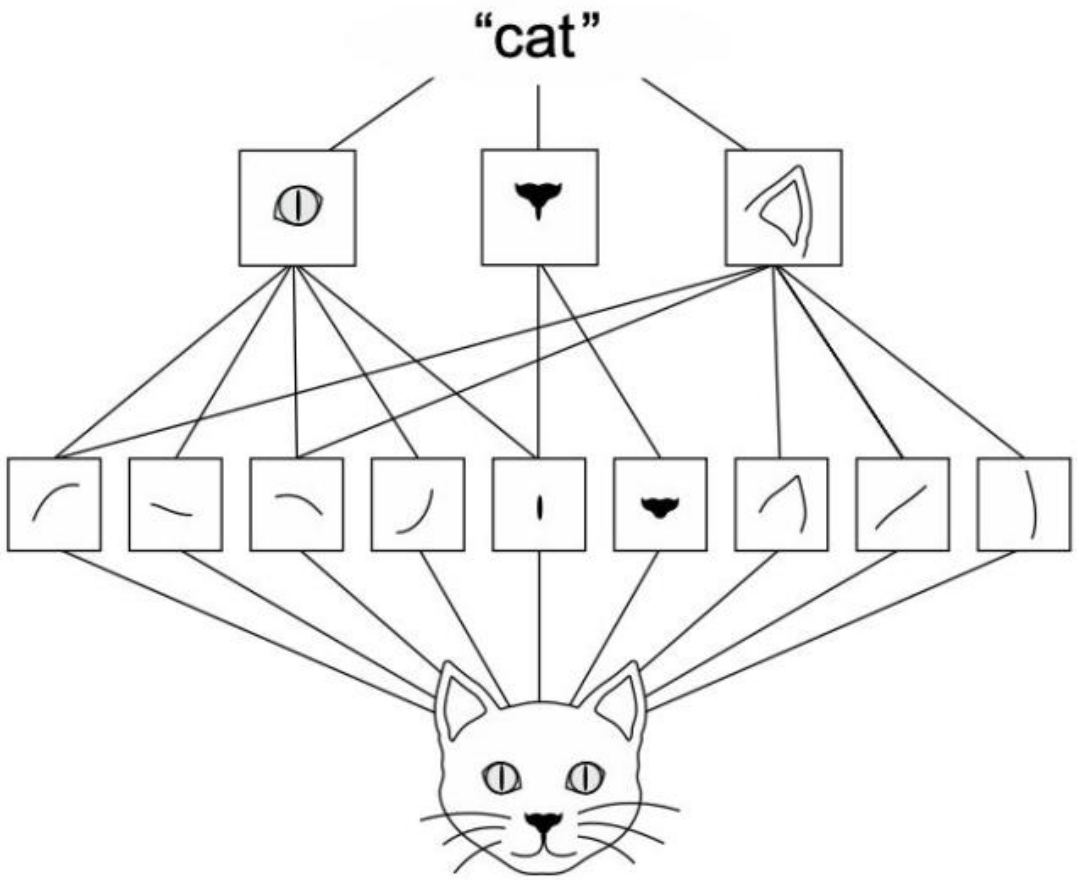
Redes neurais convolucionais



- CNNs são **mais eficientes que redes densas**: o **número de pesos depende do tamanho e da quantidade de filtros**, não da resolução da imagem.
- CNNs exploram as propriedades naturais das imagens:
  - **Localidade espacial**
  - **Estacionariedade**
  - **Composição hierárquica de padrões**

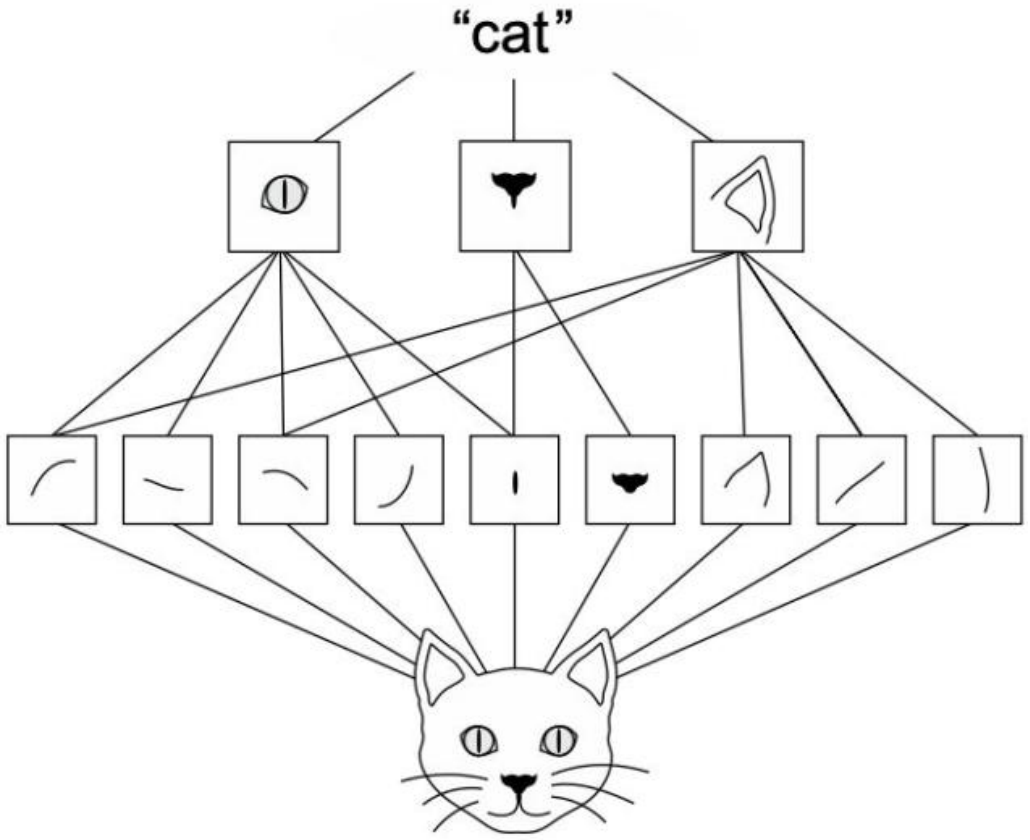
OBS.: CNNs são, em geral, usadas com imagens, mas podem ser aplicadas a outros tipos de dados (e.g., áudio)

Localidade espacial



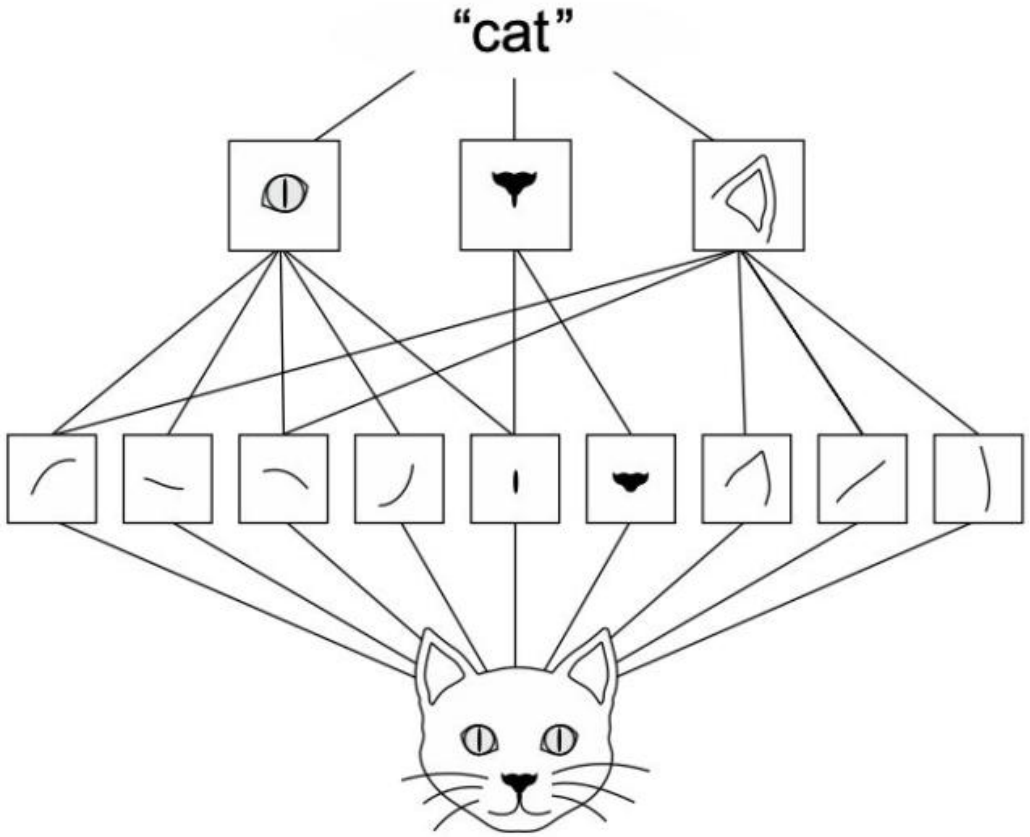
- A propriedade da **localidade espacial** diz que *pixels* próximos tendem a ser **correlacionados**.
- Por exemplo, os *pixels* dos olhos do gato.
- Ou seja, a **maior parte da informação visual** está em **pequenas regiões da imagem**.
- Como as CNNs exploram isso?

Localidade espacial



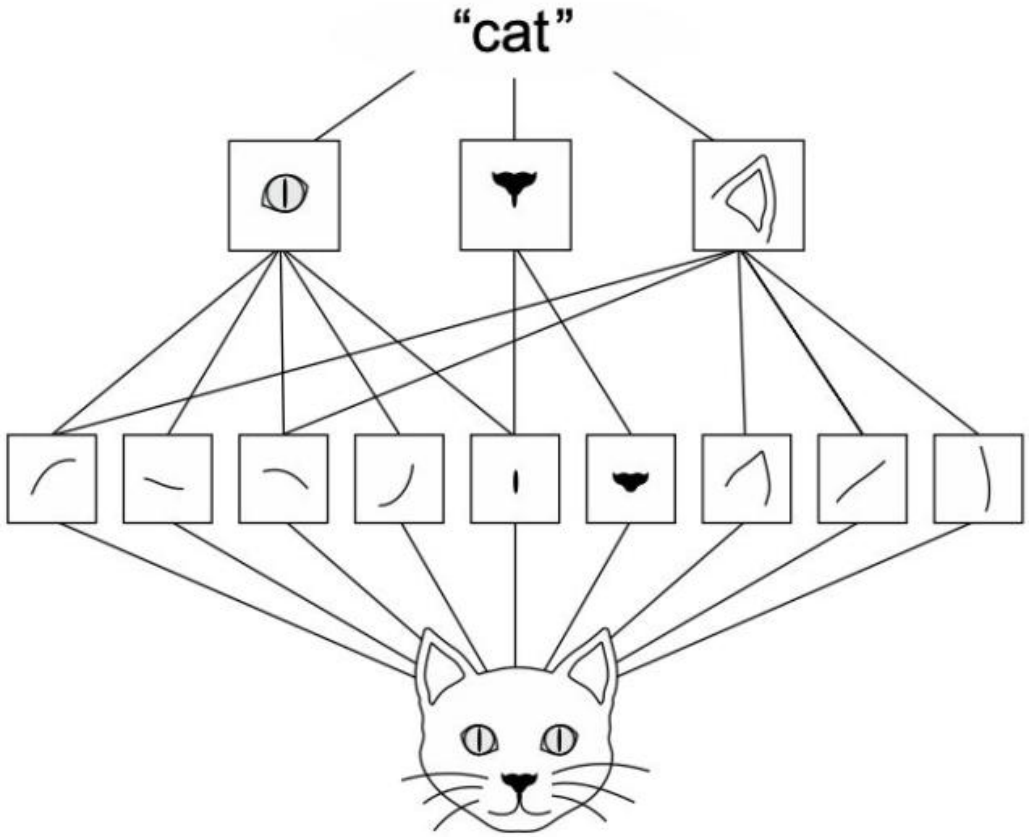
- Em vez de **conectar cada neurônio da CNN a todos os *pixels*** da imagem, cada neurônio “olha” apenas para uma **pequena região**, chamada de **campo receptivo**.
- O tamanho do campo receptivo é definido por uma matriz (2D ou 3D) chamada de **filtro (de convolução)** ou ***kernel***.
- Isso permite detectar **padrões locais** como:
  - bordas, linhas, cantos, cor
  - texturas, partes elementares de objetos
  - partes reconhecíveis de objetos: olhos ou orelhas
  - objeto inteiro: gato

Estacionariedade



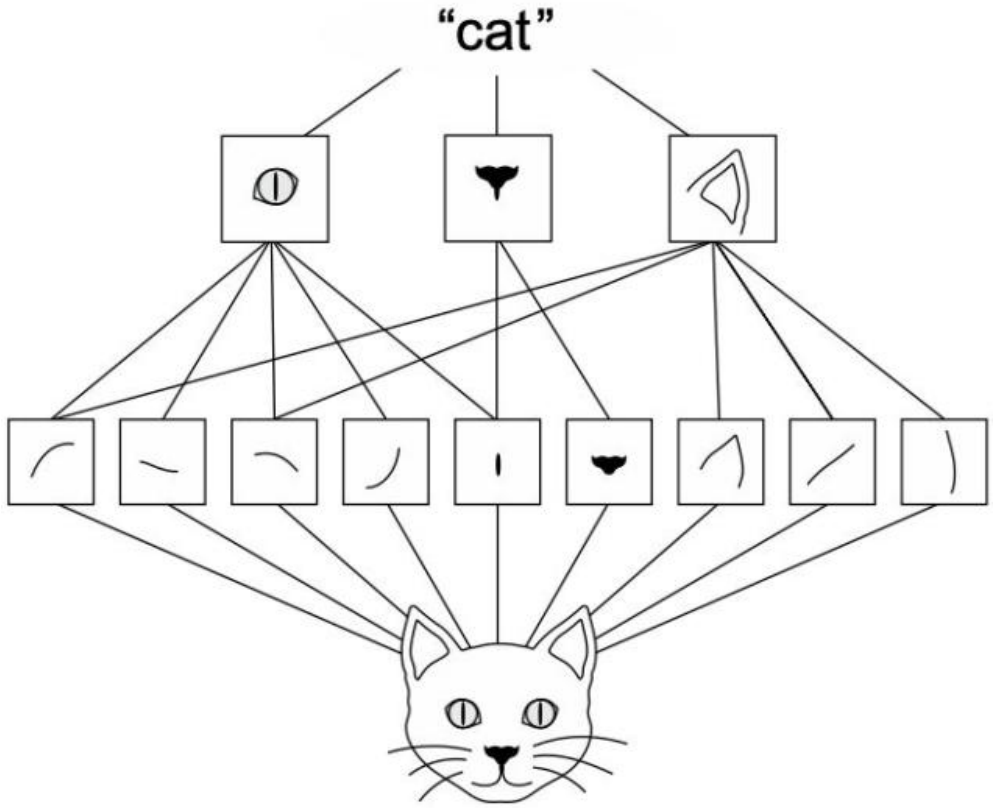
- A estacionariedade significa que o **mesmo padrão pode aparecer em qualquer posição da imagem**.
- Por exemplo:
  - olhos, orelhas e patas do gato podem aparecer em diferentes regiões da imagem
- Como as CNNs exploram isso?

Estacionariedade



- As **CNNs compartilham os pesos**.
  - O **mesmo filtro** é aplicado em **toda a imagem**.
- Por exemplo, um filtro que detecta bordas é o mesmo para todas as posições.
- Isso traz grandes **vantagens**:
  - muito menos parâmetros
  - melhora a generalização (modelo menor, menos risco de *overfitting*)
  - capacidade de detectar o mesmo padrão em qualquer posição da imagem

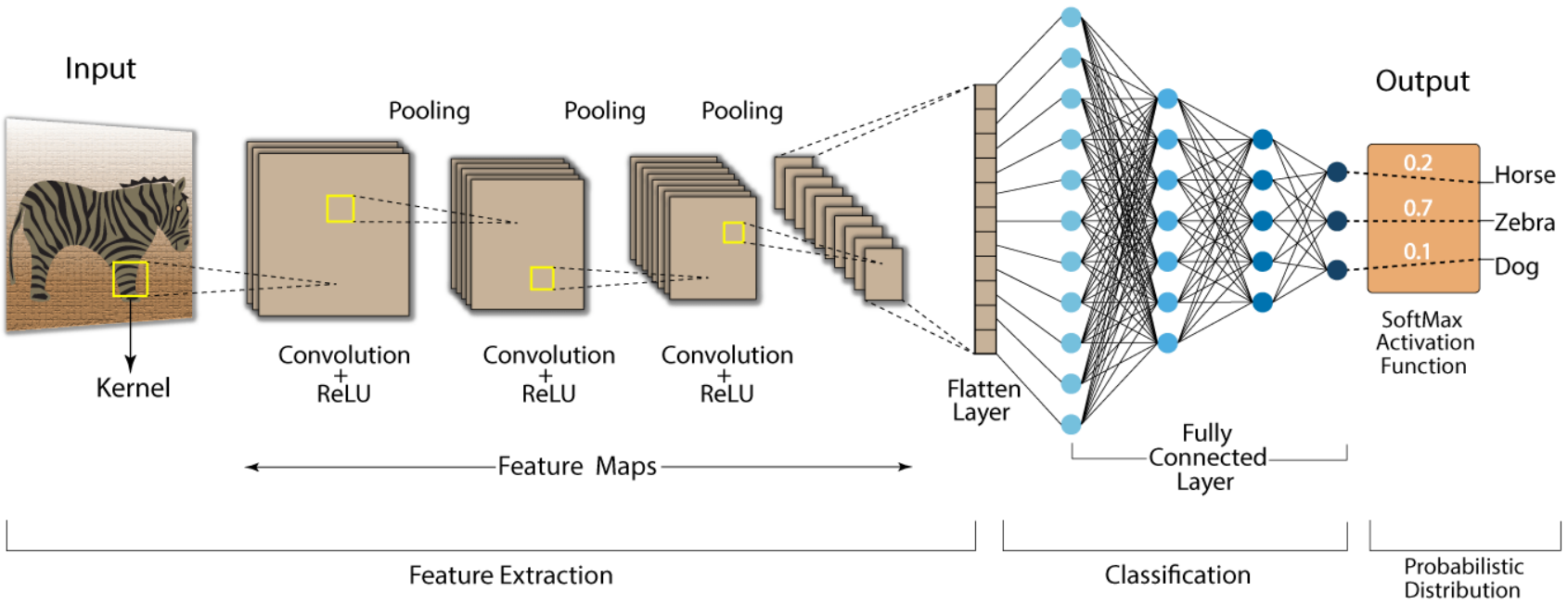
Composição hierárquica de padrões



- Imagens possuem uma **estrutura hierárquica**.
- Padrões simples formam padrões mais complexos:
  - **Simples**: contorno da orelha, borda do nariz e do olho
  - **Intermediários**: curvatura da orelha, forma triangular do nariz
  - **Partes menores**: olho, nariz, bigode, boca
  - **Partes maiores**: face, cabeça, corpo, rabo
  - **Objeto completo**: gato
- As **camadas** de uma CNN **aprendem a hierarquia** de padrões da imagem.
  - bordas → contornos → partes → objeto

Como é a arquitetura de uma CNN?

Arquitetura geral de uma CNN



- Uma **CNN para classificação de imagens** é composta por:
  - Uma entrada (imagem)
  - Um ou mais grupos de
    - Camadas de convolução
      - ✓ Filtros de convolução
      - ✓ Funções de ativação
    - Camadas de *pooling*
  - Uma camada *flatten*
  - Uma ou mais camadas totalmente conectadas
  - Uma função de ativação *softmax* para gerar as probabilidades de saída

Vamos entender um pouco mais sobre cada uma das partes das CNNs

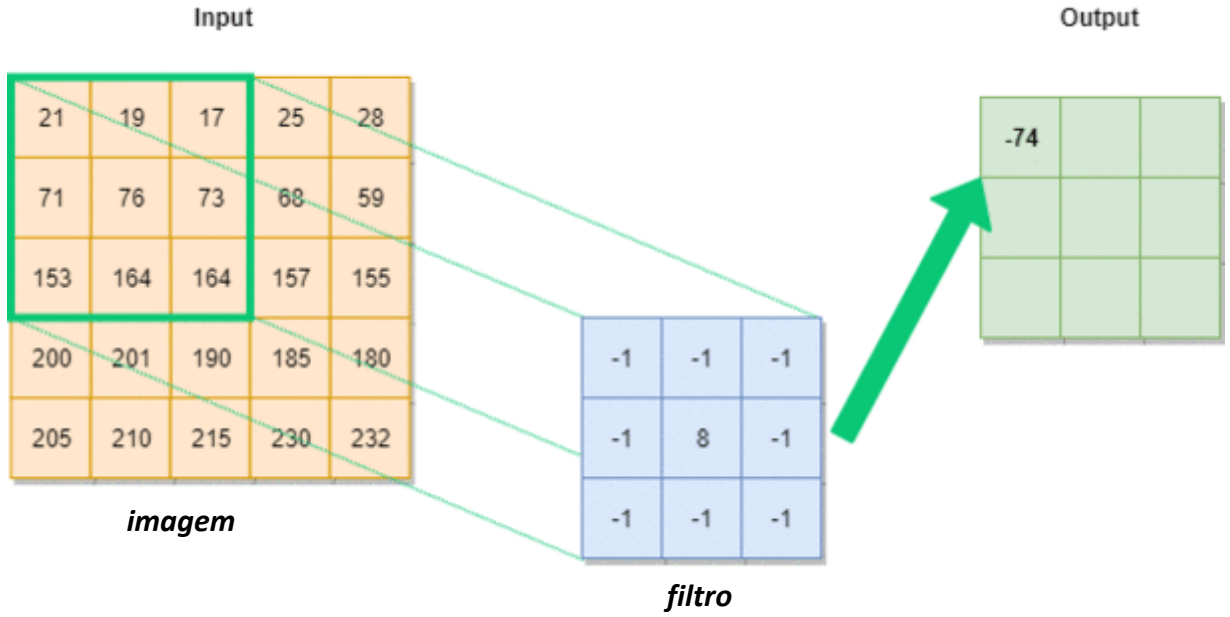
Imagens





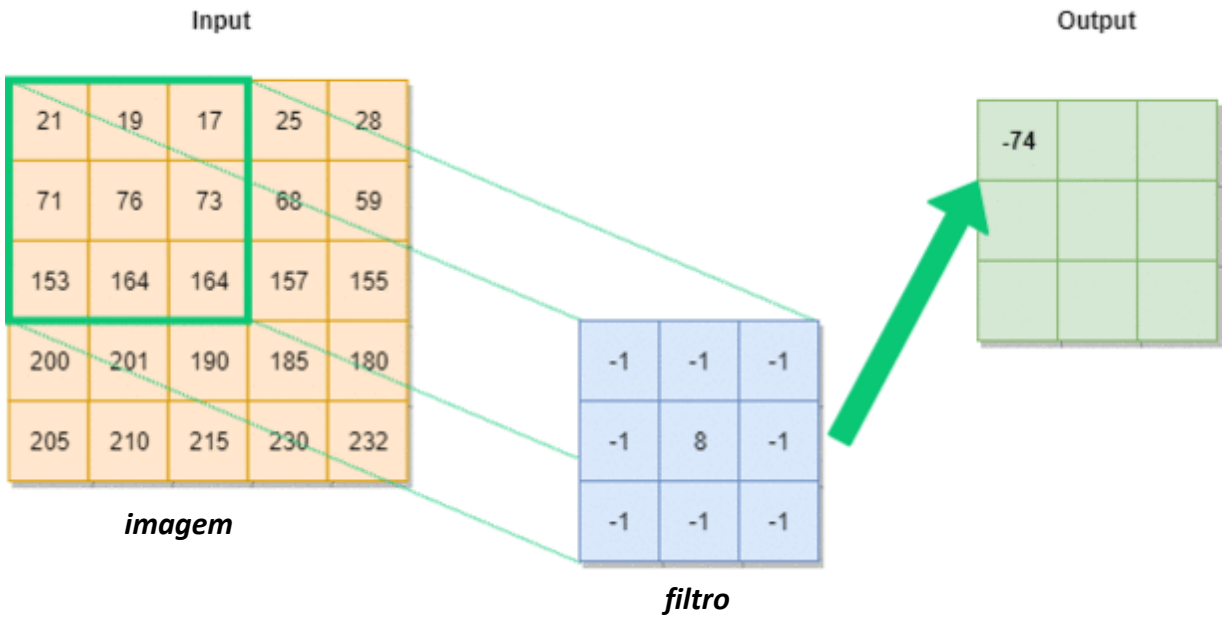
- Podemos agrupar as imagens em dois tipos: **coloridas** e em **tons de cinza**.
- Em imagens coloridas, cada *pixel* é **formado por três valores**: vermelho (R), verde (G) e azul (B).
- Imagens coloridas são **tridimensionais** (3D):
  - altura × largura × 3 canais de cores
  - Cada canal indicada a quantidade de cada cor.
- Imagens em tons de cinza são **bidimensionais**:
  - altura × largura
  - Cada *pixel* carrega apenas informação de **intensidade luminosa**

Camada de convolução



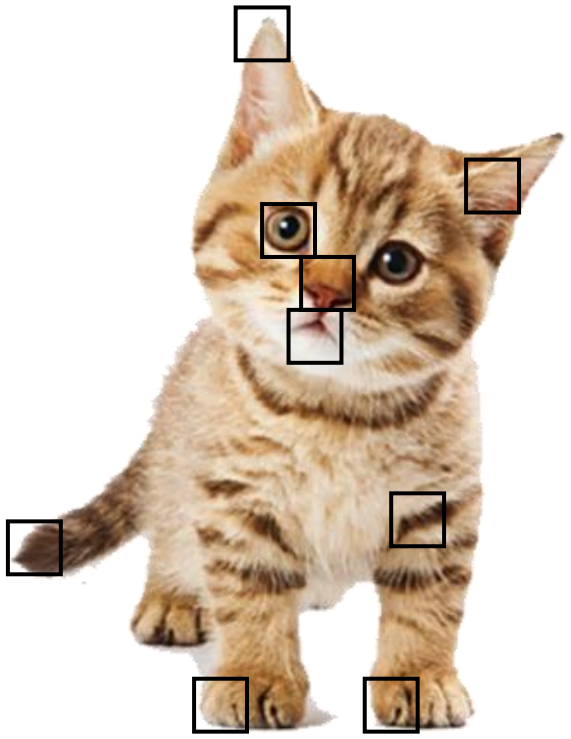
- A camada de convolução é responsável por **extrair características da imagem**.
- Ela faz isso **deslizando os filtros de convolução** sobre a imagem.
- O filtro:
  - tem seus elementos multiplicados com os valores locais da imagem (campo receptivo)
    - A multiplicação é elemento a elemento
  - soma os resultados,
  - e gera uma saída escalar.

Camada de convolução



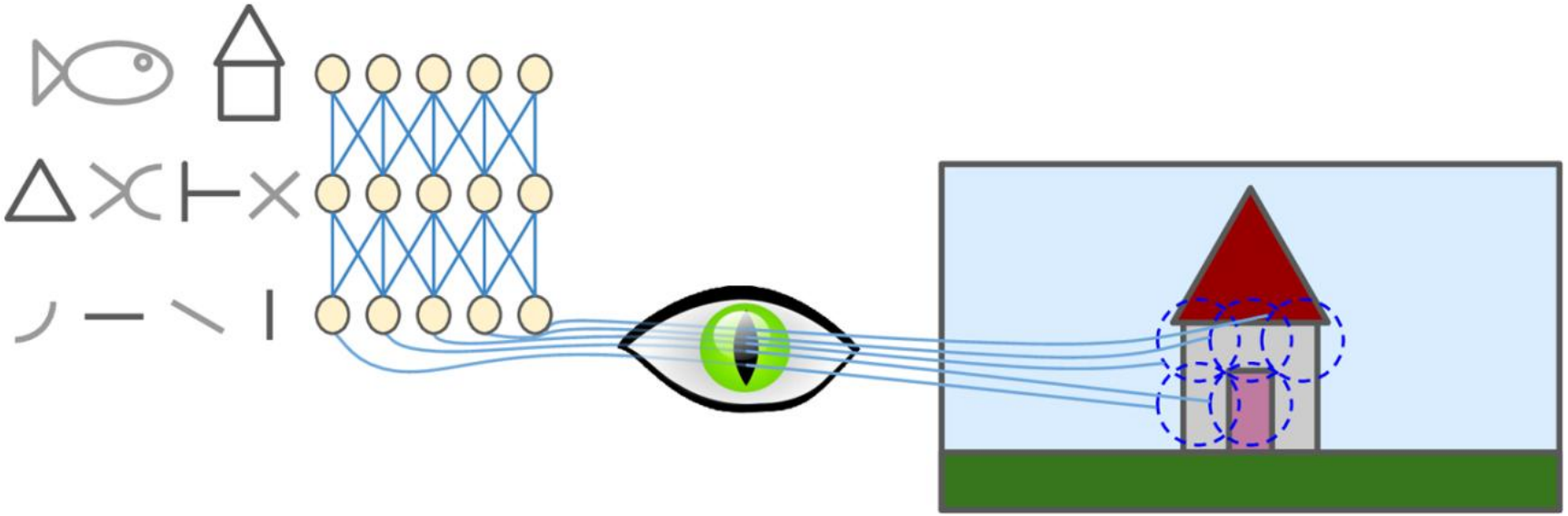
- Após o filtro, aplica-se uma **função de ativação não linear ao escalar**.
  - Em geral, usa-se a função *rectified linear unit* (ReLU): $\max(0, x)$.
- Esse processo é repetido por toda a imagem.
- O resultado é um **mapa de características** (*feature map*) **detectadas**.
- Ou seja, uma **mapa de padrões detectados**.

Filtros de convolução ou *kernels*



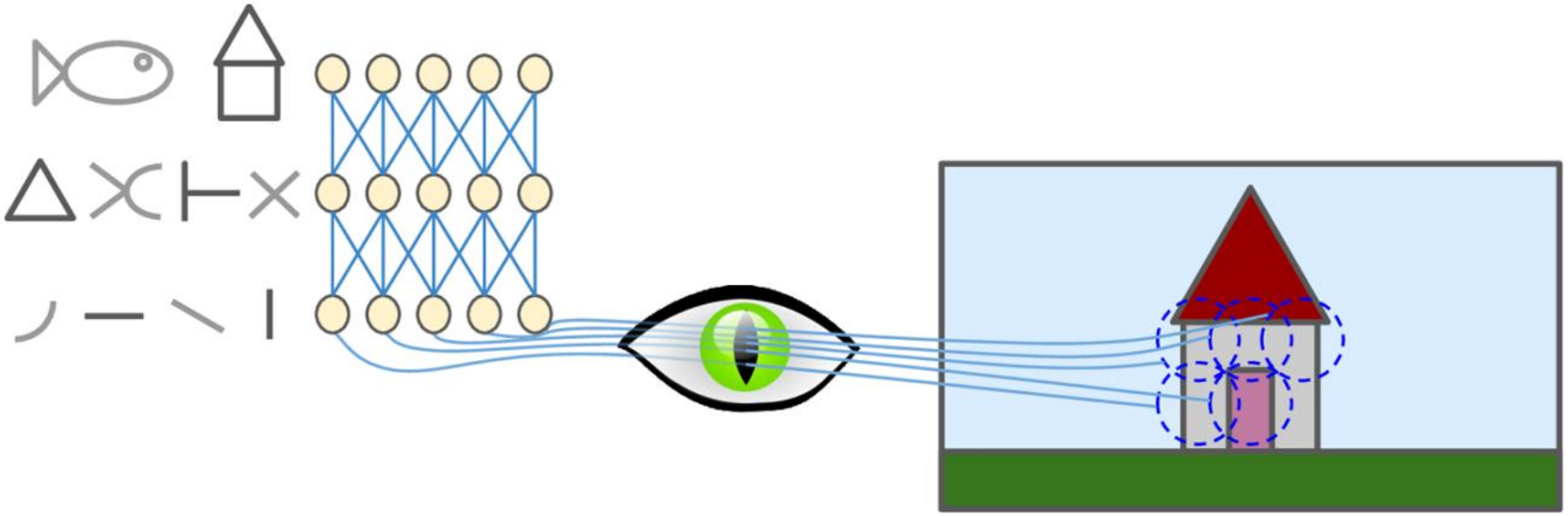
- Os **filtros** são **treinados para detectar características (padrões) presentes nas imagens**.
  - Os filtros têm **pesos que são aprendidos**.
- Essa ideia de usar **filtros** para **detectar padrões** é **baseada** em como os **neurônios biológicos do córtex visual funcionam**.

Neurônios do córtex visual



- Os **neurônios** do córtex visual **respondem a padrões específicos** em **pequenas regiões do campo visual** chamadas **campos receptivos**.
- À medida que o sinal visual percorre as camadas do cérebro, os **neurônios respondem** a **padrões mais complexos em campos receptivos maiores**.

Neurônios do córtex visual



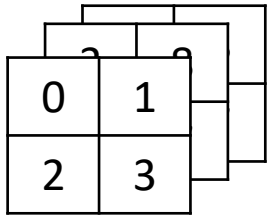
- Alguns **neurônios com campos receptivos pequenos reagem** a **linhas horizontais**, enquanto **outros, reagem** a **linhas com orientações diferentes**.
- Outros com **campos receptivos maiores** reagem a **padrões mais complexos** que são **combinações de padrões mais simples**.

Filtros de convolução ou *kernels*

Kernel 2D

0	1
2	3

Kernel 3D



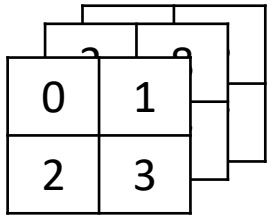
- Um ***kernel*** é uma **matriz** (ou tensor) responsável por **detectar características específicas em uma imagem**.
- Ele percorre uma imagem realizando **operações de convolução** entre seus elementos e os dos *pixels* na região da imagem correspondente ao seu **campo receptivo**.
- Para imagens **coloridas**, eles têm **3 dimensões** e **2 dimensões** para imagens em **tons de cinza**.

Filtros de convolução ou *kernels*

Kernel 2D

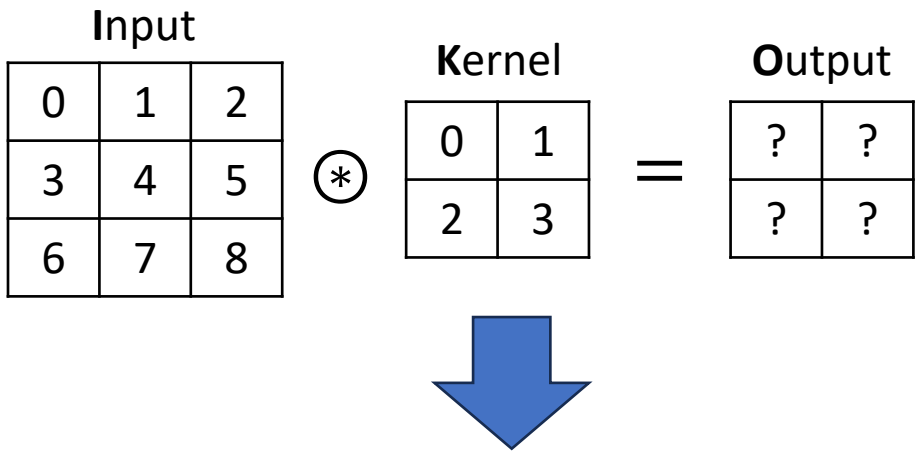
0	1
2	3

Kernel 3D



- Cada ***kernel*** **detecta um tipo particular de característica**, como bordas, texturas, partes específicas de objetos e objetos inteiros.
- Os ***kernels*** **aprendem** a **detectar** as **características mais relevantes** para a tarefa em mãos.
  - Eles **aprendem**, a partir do **conjunto de treinamento**, os valores dos **elementos das matrizes** necessários para **detectar as características que identificam, por exemplo, um gato**.

Operação de convolução com *kernel* 2D

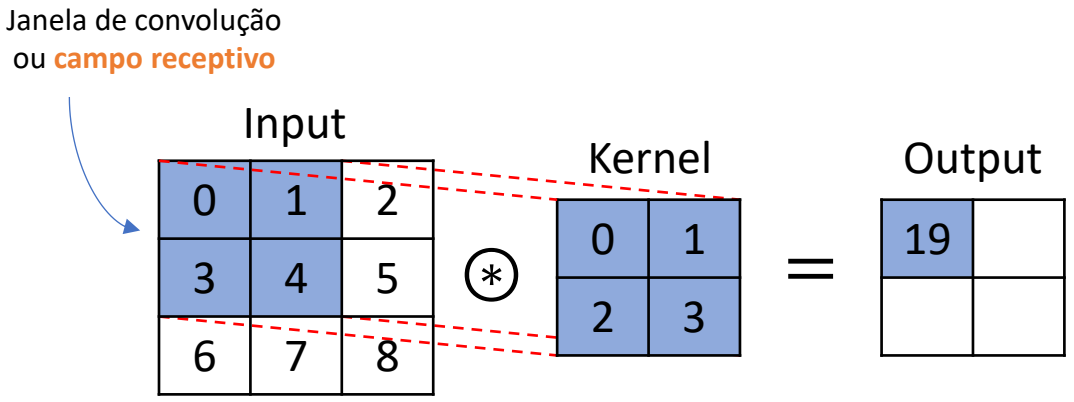


$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

I : entrada (e.g., uma imagem ou o mapa da camada anterior)
K: *kernel*
O: saída da convolução
i, j: índices da linha e da coluna na saída e no campo receptivo de entrada
m, n: índices da linha e da coluna do kernel, respectivamente

- Para entendermos como a operação de convolução funciona, vamos considerar, inicialmente, imagens bidimensionais (e.g., imagens em tons de cinza).
- O símbolo ⊛ representa a operação de convolução.
- O operação é representada pela equação ao lado.

Operação de convolução com *kernel* 2D



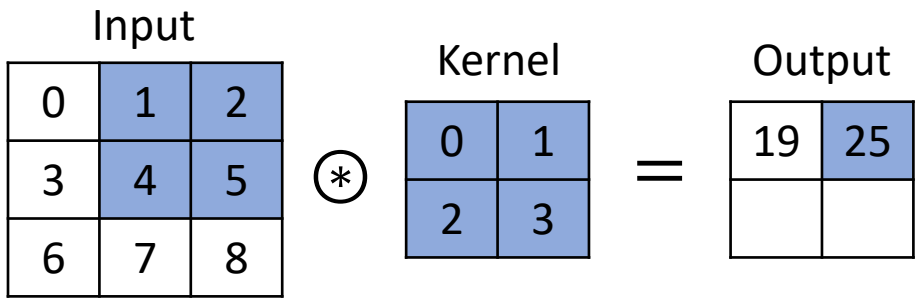
$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 0 * 0 + 1 * 1 + 3 * 2 + 4 * 3$$

$$= 19$$

- Ao calcular a convolução, começamos com a **janela de convolução (ou campo receptivo)** no **canto superior esquerdo** do **tensor de entrada** (e.g., uma imagem ou saída da camada anterior).

Operação de convolução com *kernel* 2D



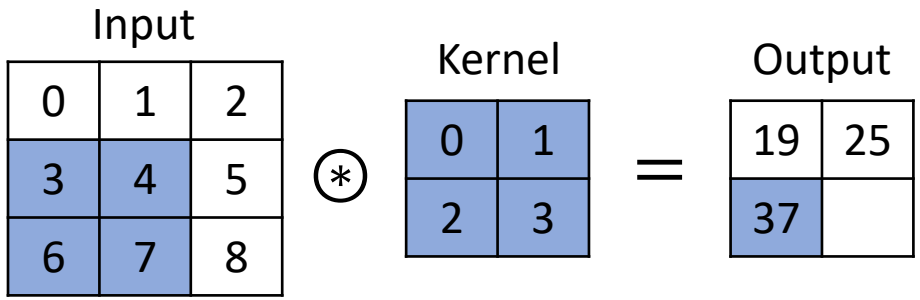
- Em seguida, deslizamos a janela, por exemplo, um elemento para a direita.
- O deslocamento é definido pelo ***stride***, o qual discutiremos a seguir.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 1 * 0 + 2 * 1 + 4 * 2 + 5 * 3$$

$$= 25$$

Operação de convolução com *kernel* 2D



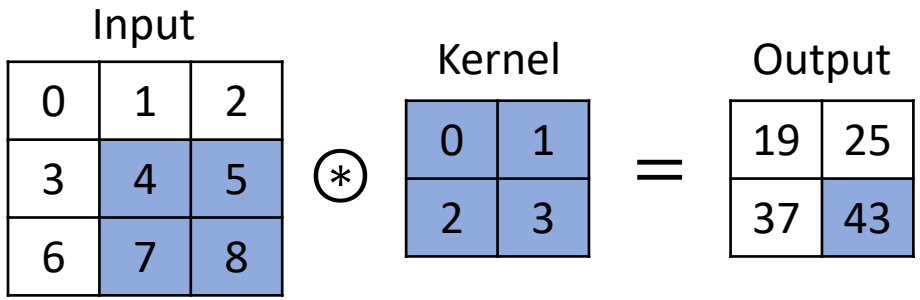
- Ao chegar-se ao final das colunas do tensor de entrada, volta-se ao seu início, deslizando a janela, por exemplo, um elemento para baixo, ou seja, uma linha.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 3 * 0 + 4 * 1 + 6 * 2 + 7 * 3$$

$$= 37$$

Operação de convolução com *kernel* 2D



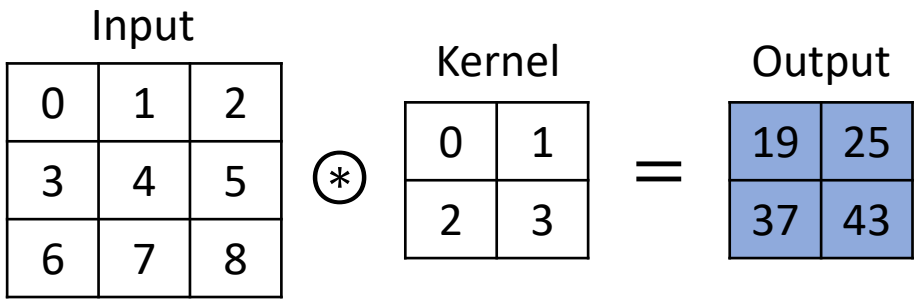
- Em seguida, deslizamos a janela um elemento para a direita.
- Esse processo se repete até que a janela de convolução tenha percorrido todo o tensor de entrada.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 4 * 0 + 5 * 1 + 7 * 2 + 8 * 3$$

$$= 43$$

Mapa de características

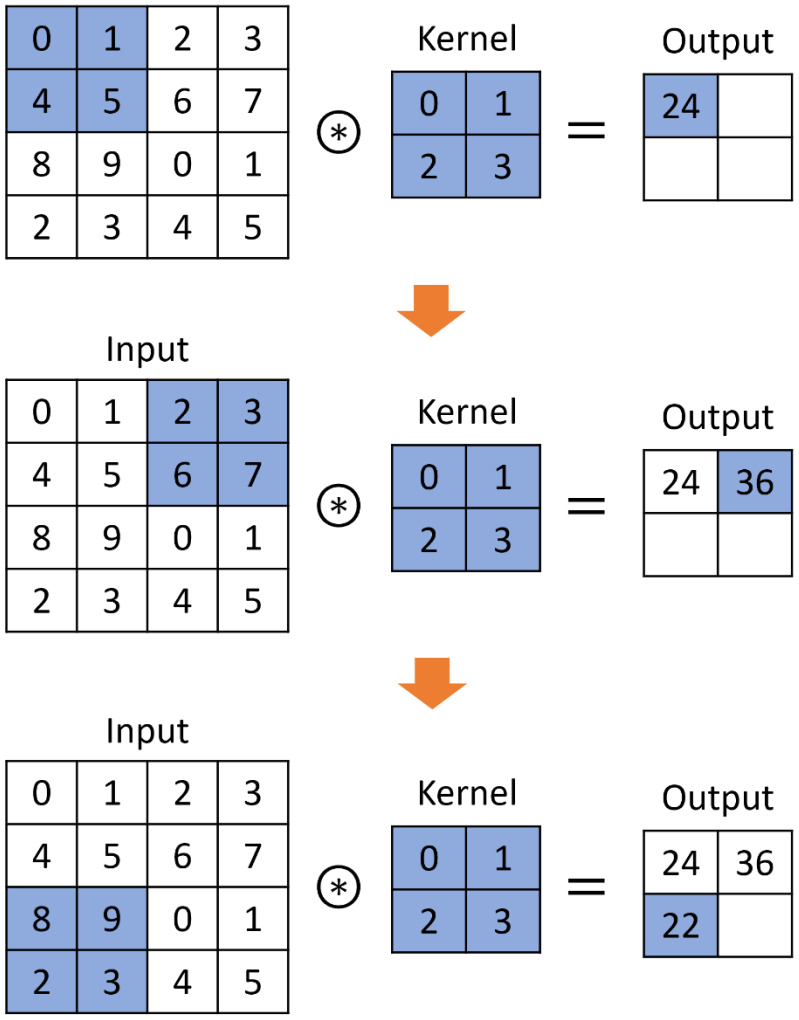


- Lembrando que o **objetivo** dos ***kernels*** é **identificar características na entrada**.
- Portanto, o resultado da operação de convolução é chamado de **mapa de características ou mapa de ativações.**
- É chamado assim, pois **fornece as ativações de um filtro ao longo de cada posição espacial** do tensor de entrada.
- O mapa mostra **onde** e com que **intensidade** uma característica específica (detectada pelo filtro) está presente no tensor de entrada.

Stride

- O ***stride*** controla o deslizamento do *kernel* sobre o tensor de entrada.
- É o número de *pixels* que o *kernel* avança a cada aplicação da convolução.
- No exemplo anterior, o ***stride*** era de 1 *pixel* para a direita e para baixo.
- Para que serve sua variação?
  - Reduzir o tamanho do mapa de características (*downsampling*)
    - Aumentar o *stride* **reduz a resolução da saída.**
  - Reduzir o custo computacional.
    - Aumentar o *stride* **reduz a quantidade de cálculos.**
  - Controlar o nível de detalhe preservado.
    - *Stride* pequeno: maior sobreposição entre janelas, o que preserva mais resolução espacial e informação de localização na saída, porém, tem custo computacional maior.
    - *Stride* grande: saída de menor resolução, menor custo computacional, mas com risco de perder detalhes finos (o *kernel* pula posições) e a localização fica menos precisa.

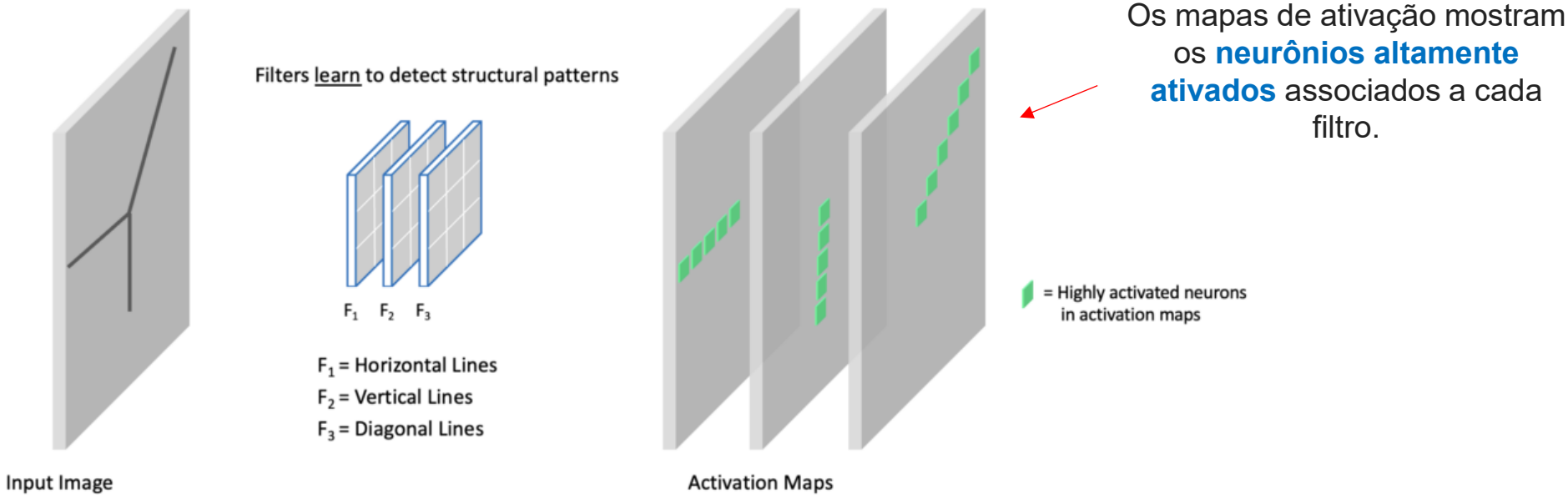
Stride



- Portanto, o *stride* controla o equilíbrio entre **resolução espacial vs. eficiência.**
- No exemplo ao lado, o *stride* é de 2 para deslizamentos ao longo das colunas e linhas.
  - *Downsampling*: Entrada 4×4 tem como saída um mapa de 2×2
- Porém, ele pode ser diferente para deslocamentos ao longo das linhas e colunas.

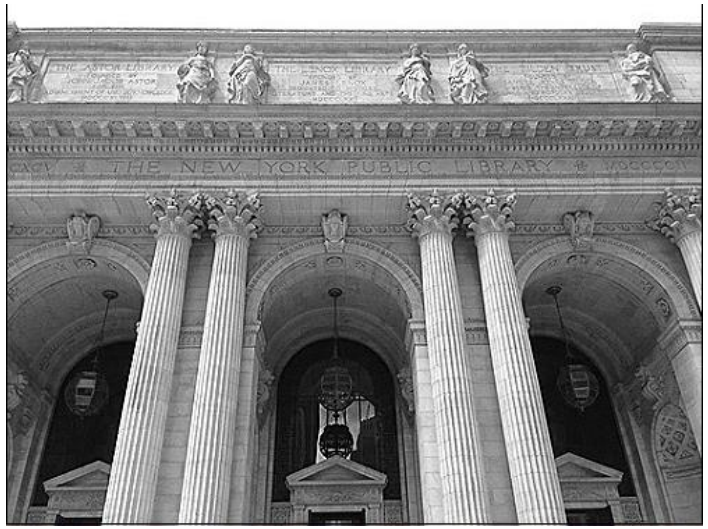
Filtros detectam padrões

Filtros detectam padrões



- Na figura, a imagem de entrada apresenta três padrões lineares (linhas).
- Temos uma camada convolucional com três filtros.
- Cada **filtro aprende** a **detectar um padrão específico**: linhas horizontais, linhas verticais e linhas diagonais.
- **Cada filtro gera um mapa de características diferente**.

Aplicando *kernels* a imagens



kernel

-1	0	1
-2	0	2
-1	0	1

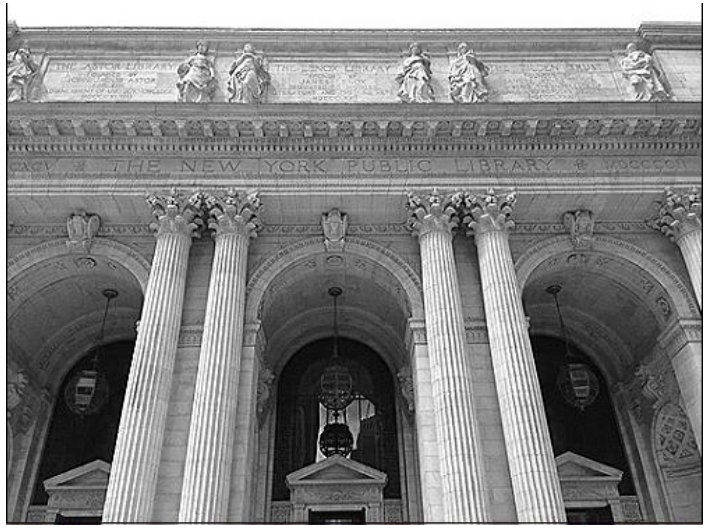
right sobel



OBS.: após a aplicação do filtro, a imagem da direita tem resolução menor do que a da esquerda

- Considerem a imagem em tons de cinza à esquerda.
- Ao aplicarmos o *kernel* mostrado, obtemos o resultado à direita.
- Ele **realça muito as linhas verticais e escurece todo o resto**.
- Portanto, podemos dizer que este *kernel* é um **detector de linhas verticais**.

Aplicando *kernels* a imagens



kernel

-1	-2	-1
0	0	0
1	2	1

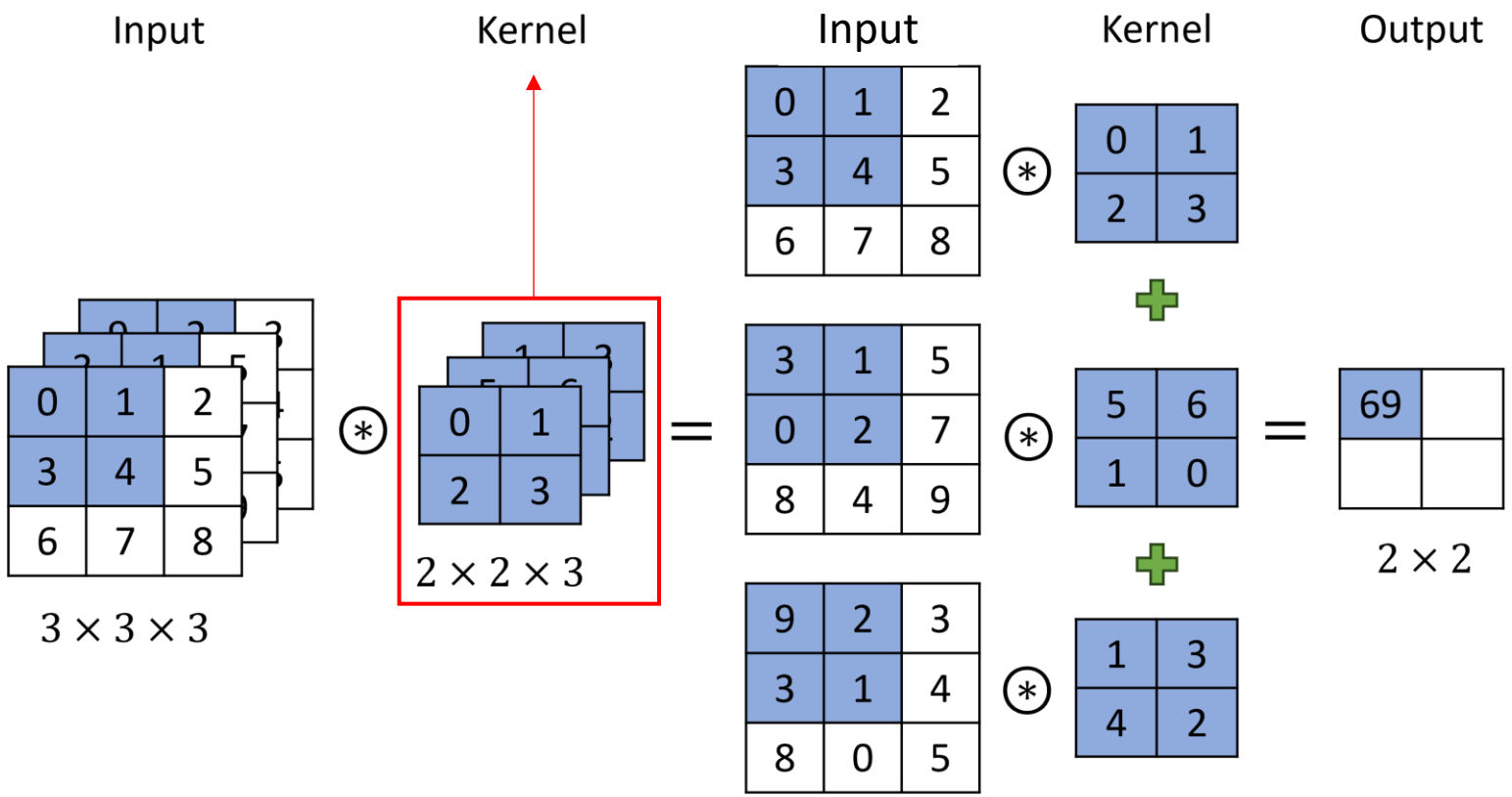
bottom sobel



- Já o *kernel* acima, **detecta linhas horizontais**, **escurecendo** quase tudo na imagem que não seja uma linha horizontal.
- Ao aplicar *kernels* como esses, **removemos quase tudo**, **exceto uma característica distinta**.
- Esse é o processo chamado de **extração de características**.
  - Processo que determina as partes mais importantes de uma imagem para um *kernel*.

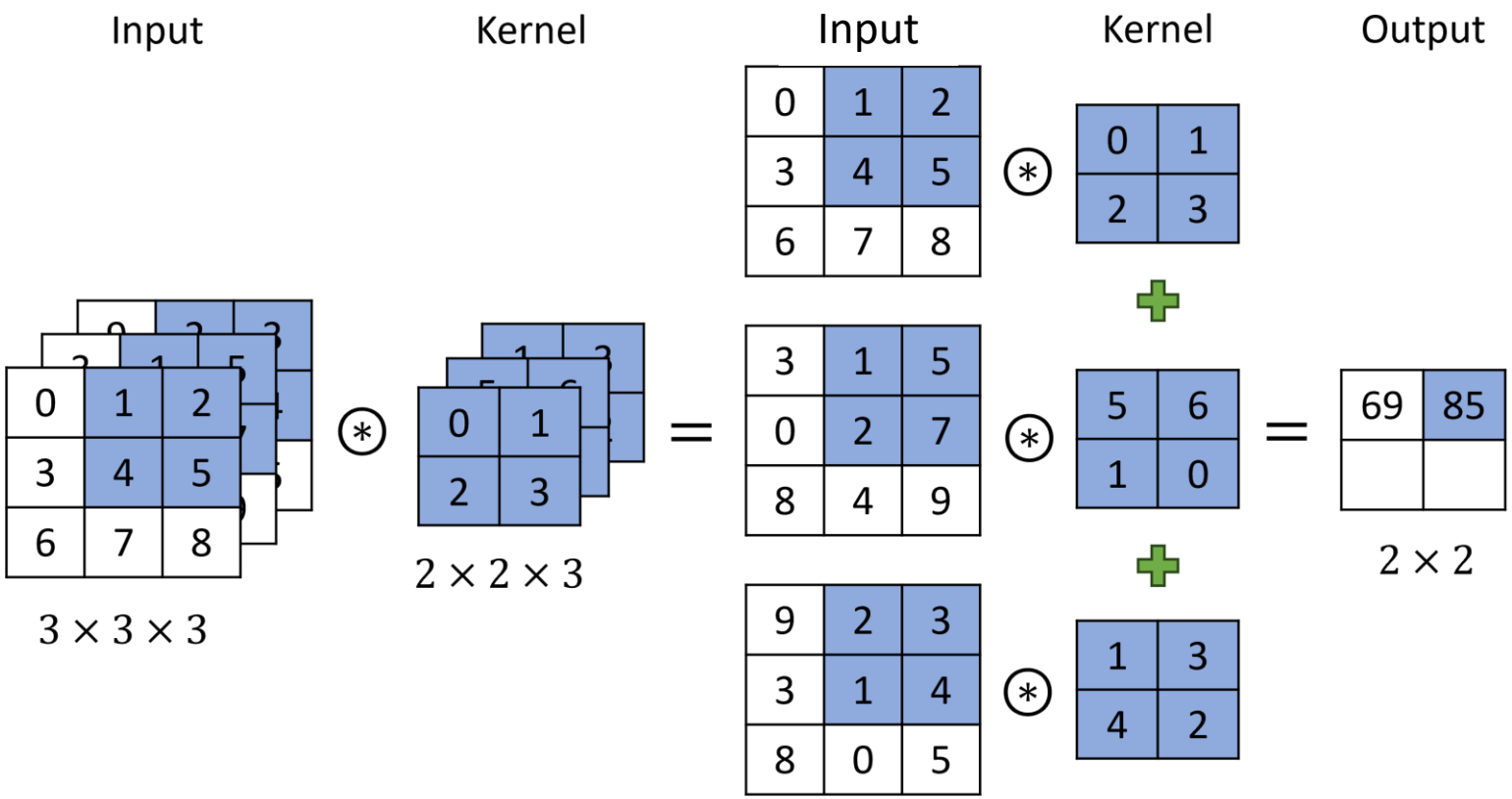
fonte: https://setosa.io/ev/image-kernels/

Operação de convolução com *kernel* 3D



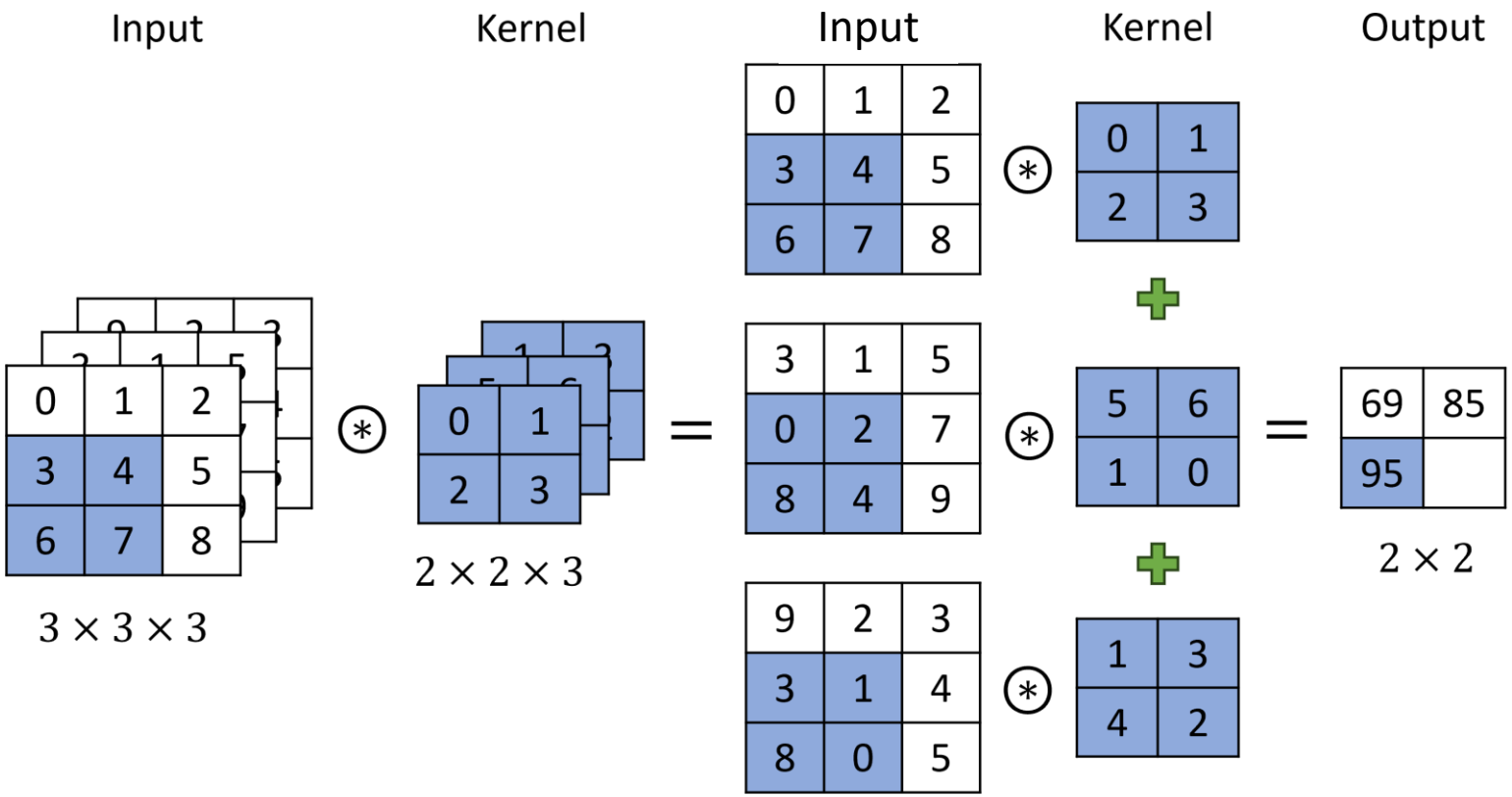
- Em geral, se a imagem tem 3 dimensões, o ***kernel*** também terá 3 dimensões.
- Nesse caso, a **convolução é aplicada por canal**.
- Ou seja, realiza-se **3 operações de convolução separadas** que têm seus **resultados somados ao final** para gerar **uma saída escalar**.
- No exemplo, usamos *stride* = 1.

Operação de convolução com *kernel* 3D

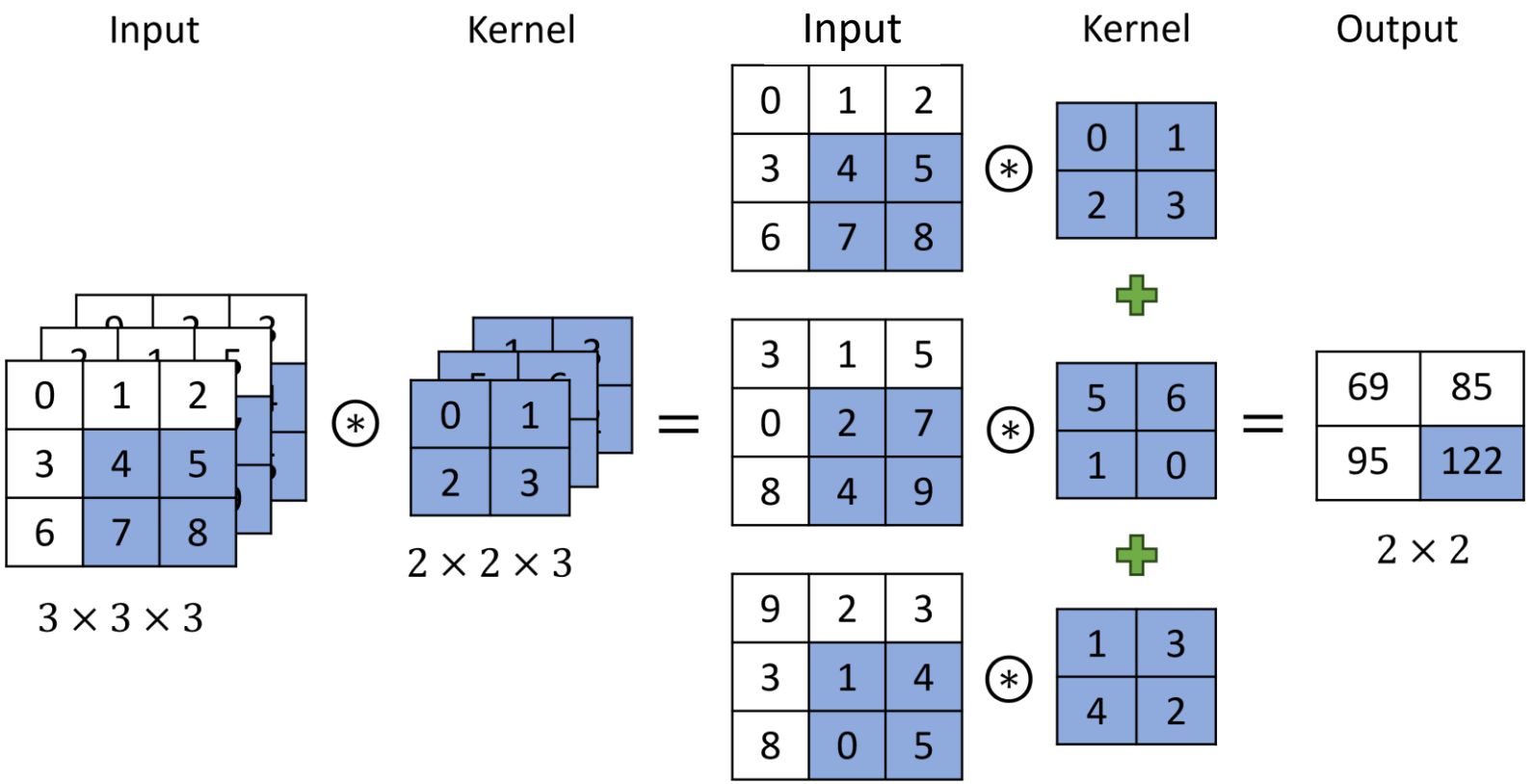


- A seguir, move-se a janela de convolução 1 elemento para a direita e calcula-se as 3 convoluções.
- Ao final, soma-se os 3 escalares resultantes das 3 convoluções para gerar o elemento do mapa de ativações.

Operação de convolução com *kernel* 3D

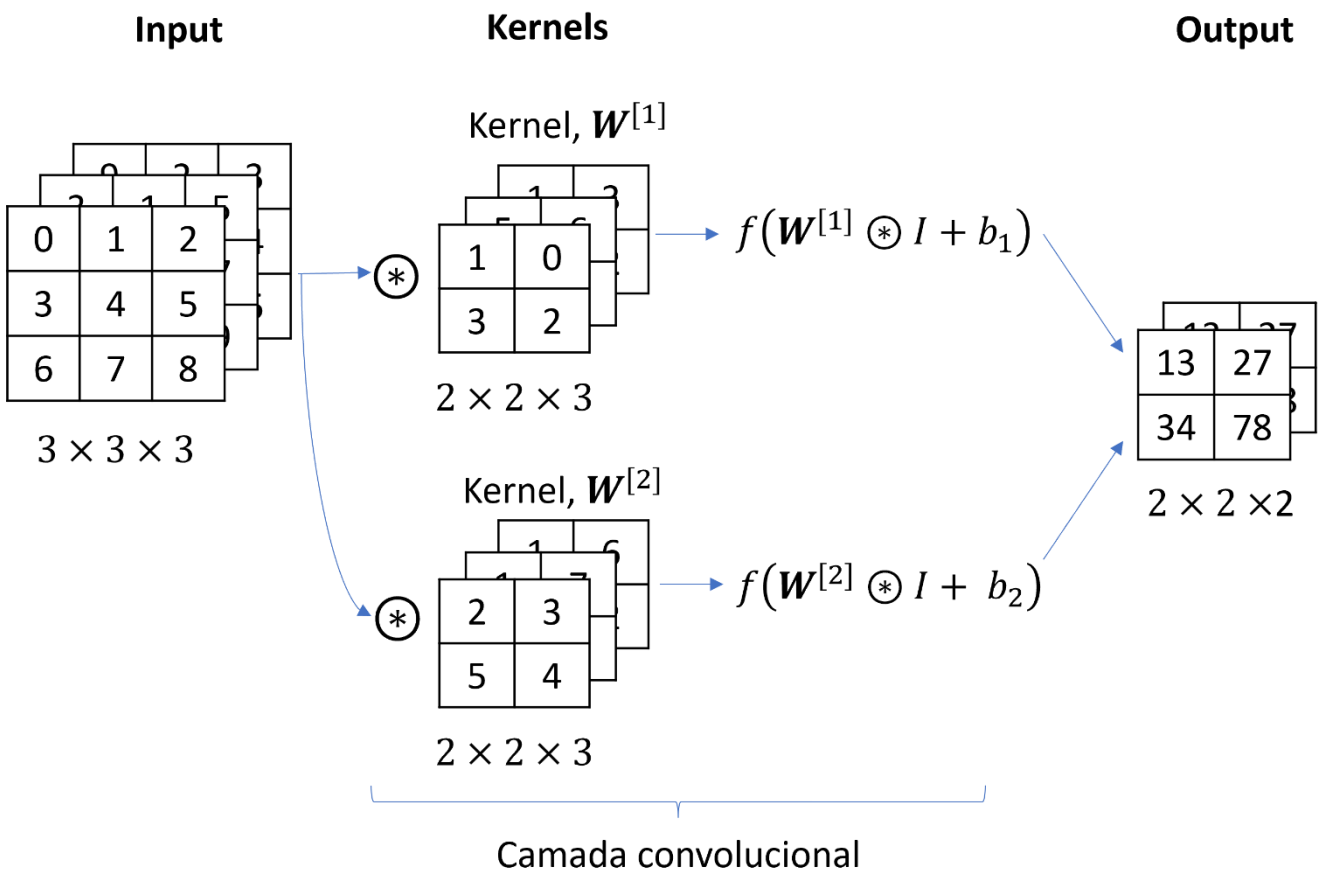


Operação de convolução com *kernel* 3D



- Move-se a janela e repete-se as convoluções até que toda a entrada seja processada.
- Ao final temos **apenas um mapa de características**.

Kernels diferentes para características diferentes



- Em geral, cada **camada convolucional possui vários *kernels***.
- Cada ***kernel* detecta uma característica diferente**.
- O resultado da **convolução com cada *kernel* tem um valor de *bias* somado** a ele e o resultado é passado por uma função de ativação, $f(.)$.
- A saída da camada é o resultado do **empilhamento** dos mapas de características de todos os *kernels* da camada.

Função de ativação

Feature map, M

-1.5	10
22	-3.7



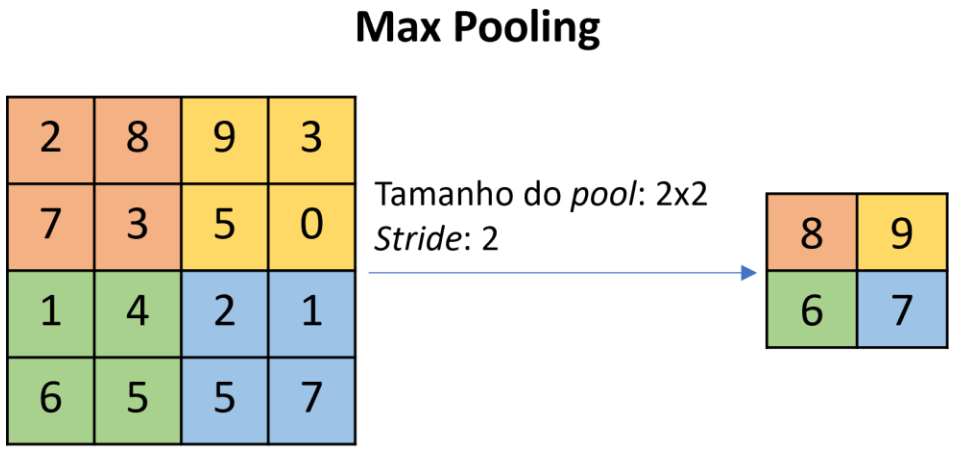
$f(M) = \max(0, M)$



0	10
22	0

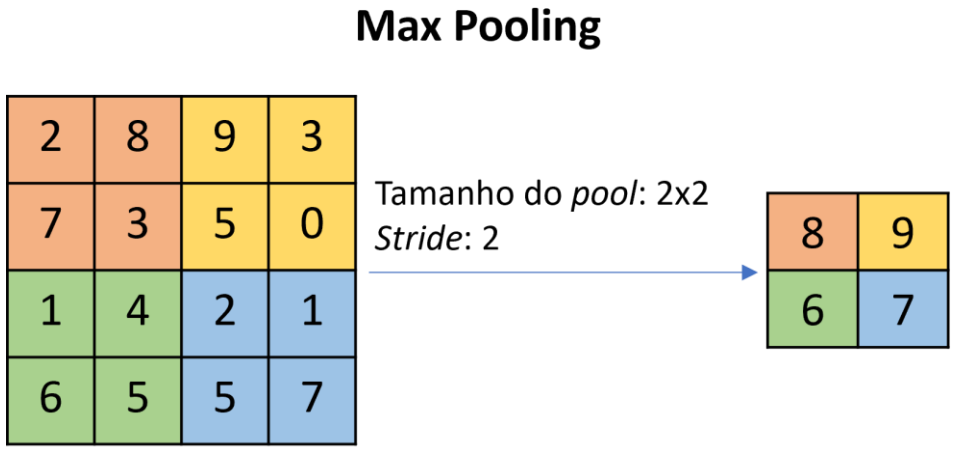
- A função de ativação introduz não linearidade.
  - Com ela, a rede aprende **relações complexas e não lineares entre as características**.
- Ela é aplicada diretamente ao mapa de características **elemento a elemento**.
- Ou seja, cada valor do mapa é transformado individualmente pela função de ativação.
- Em geral, como **CNNs tendem a ter muitas camadas**, para evitar o **desaparecimento do gradiente**, se usa funções de ativação do tipo ReLU ou suas variantes.

Camada de *Pooling* (ou subamostragem)



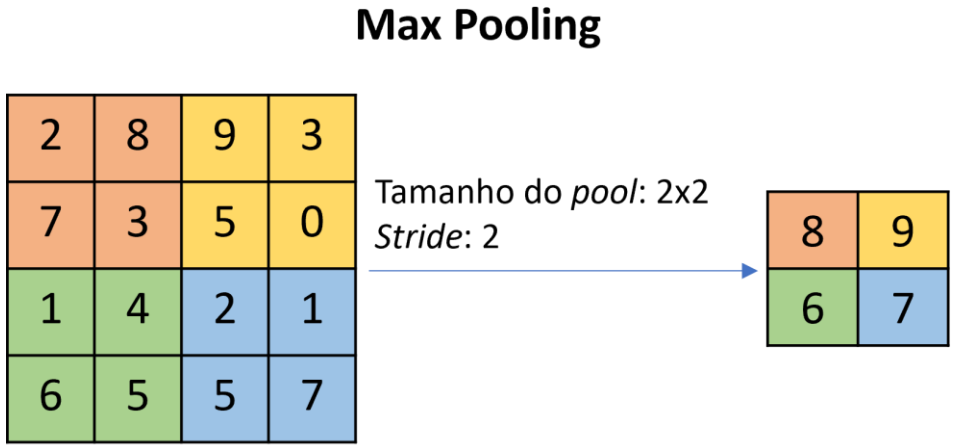
- Aplicada **após uma camada de convolução**.
- Ela **subamostra** sua entrada: **reduz o tamanho espacial** dos mapas de características.
- A operação de *pooling* **varre o mapa de características** em **pequenas regiões** (janelas) e **resume cada região a um único valor**.
- O exemplo ao lado usa uma janela de *pooling* 2×2, um *stride* de 2 elementos e seleciona o **maior valor da janela** (*max pooling*).

Camada de *Pooling* (ou subamostragem)



- O *pooling* **diminui custo computacional e a quantidade de memória usada**, assim como o *stride* que vimos antes.
- A redução das dimensões **ajuda a controlar o *overfitting***.
- **Campo receptivo maior**: com mapas menores, os *kernels* das camadas seguintes **cobrem uma área maior da imagem original**, o que ajuda a capturar padrões mais globais.

Camada de *Pooling* (ou subamostragem)

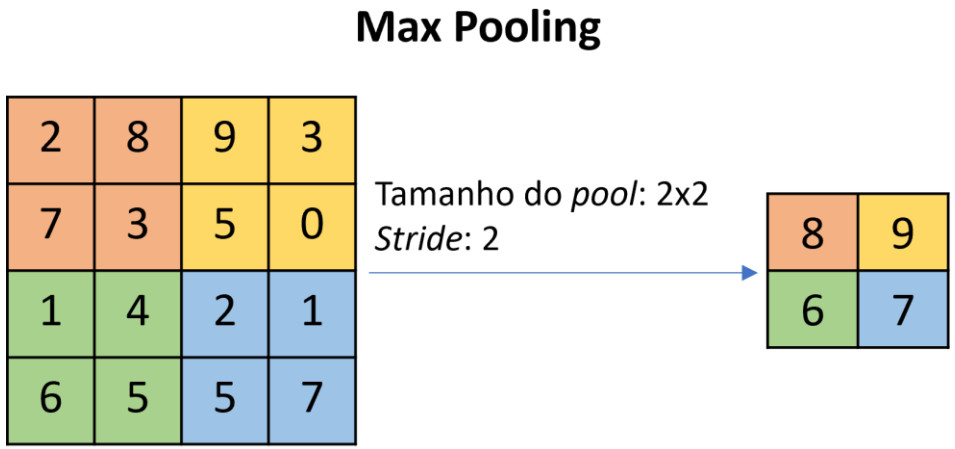


- A operação de *pooling* cria **invariância local:**
  - **Pequenos deslocamentos** dos padrões não afetam muito o resultado.
  - **Faz com que a rede foque na presença de um padrão, e não na sua localização**.
- O padrão pode:
  - estar deslocado levemente
  - ter pequenas deformações
  - ter ruído

e mesmo assim a CNN continua o reconhecendo.

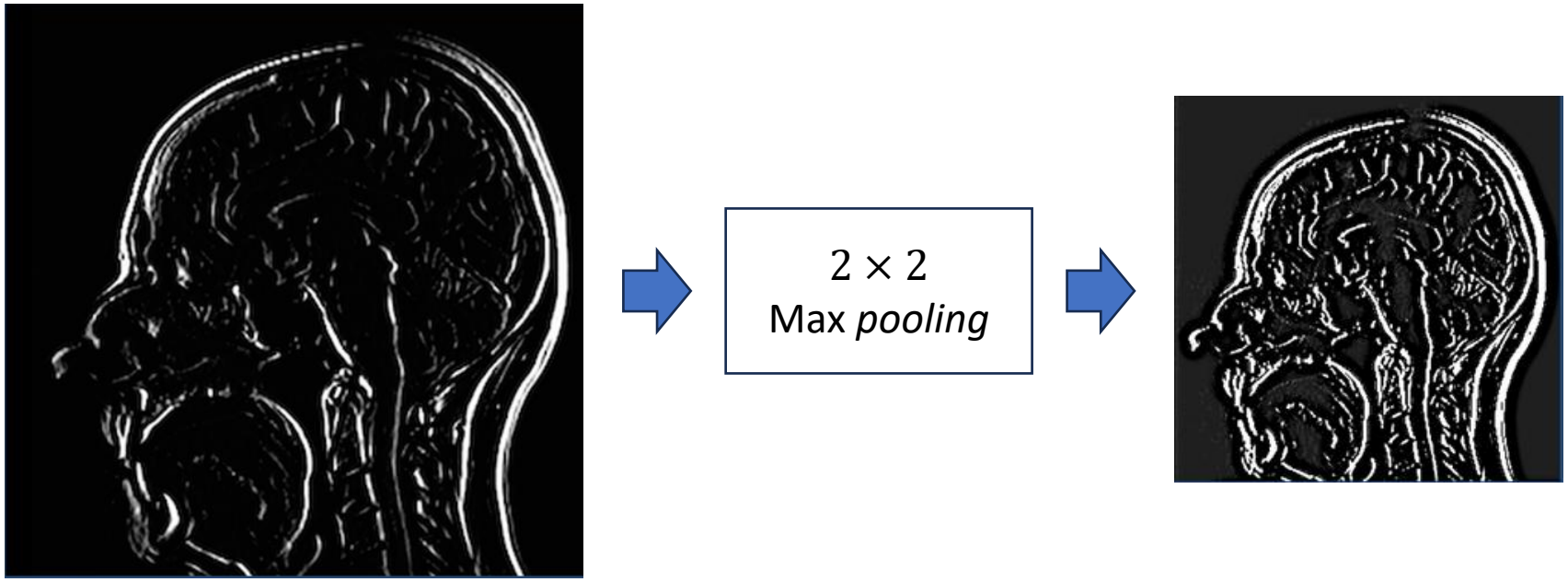
- O *pooling* contribui para que a CNN **perca informação espacial** e ganhe **informação semântica**.

Max pooling



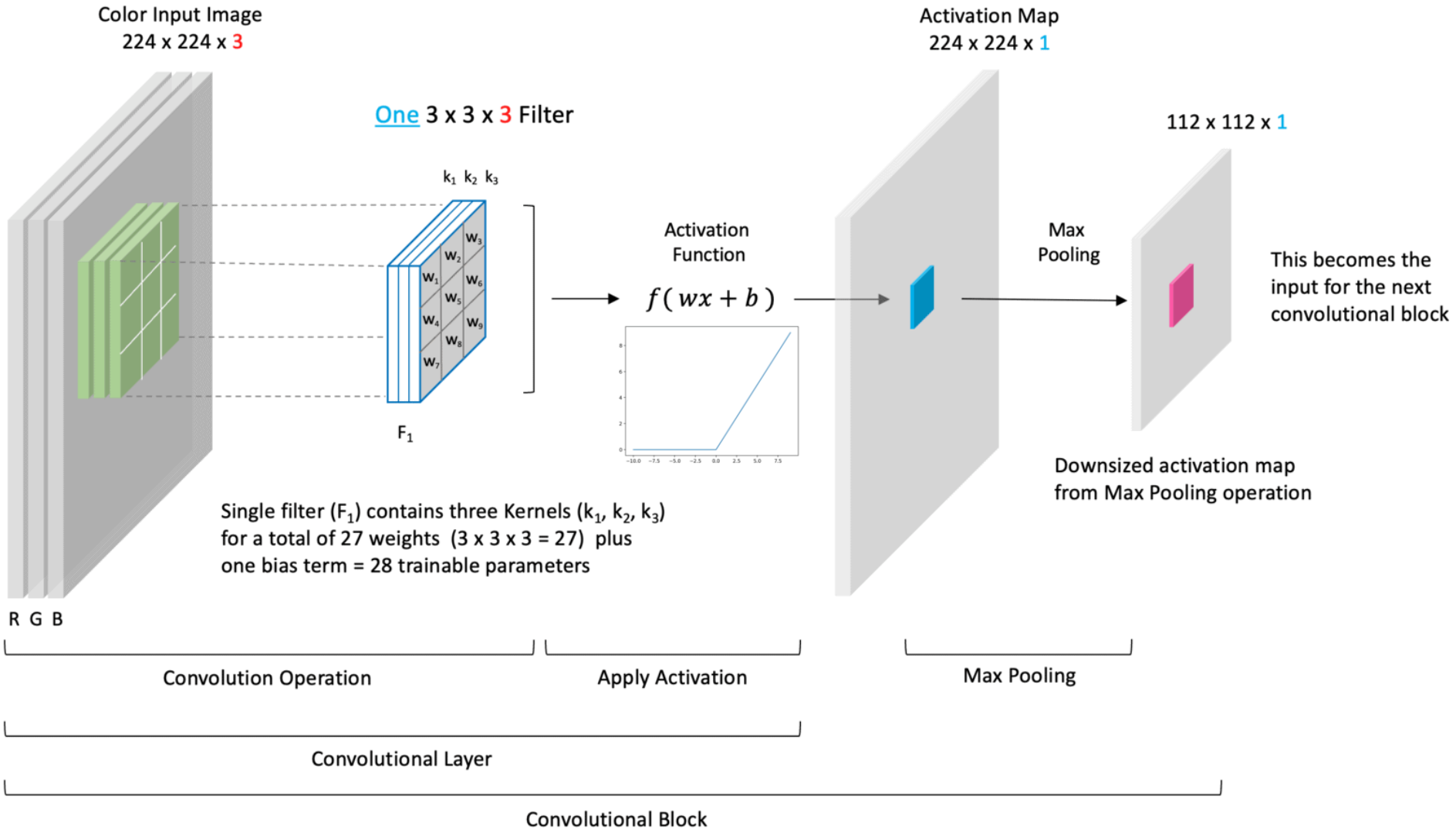
- A subamostragem mais usada é o *max pooling*, que aplica a **operação $\max(.)$** a cada mapa de características de uma camada de convolução.
- Somente o **valor máximo de cada janela passa para a próxima camada**, enquanto os outros **são descartados**.
- Ele mantém apenas **as características mais importantes** (mais fortes), **descartando as menos relevantes ou ruidosas**.

Max pooling

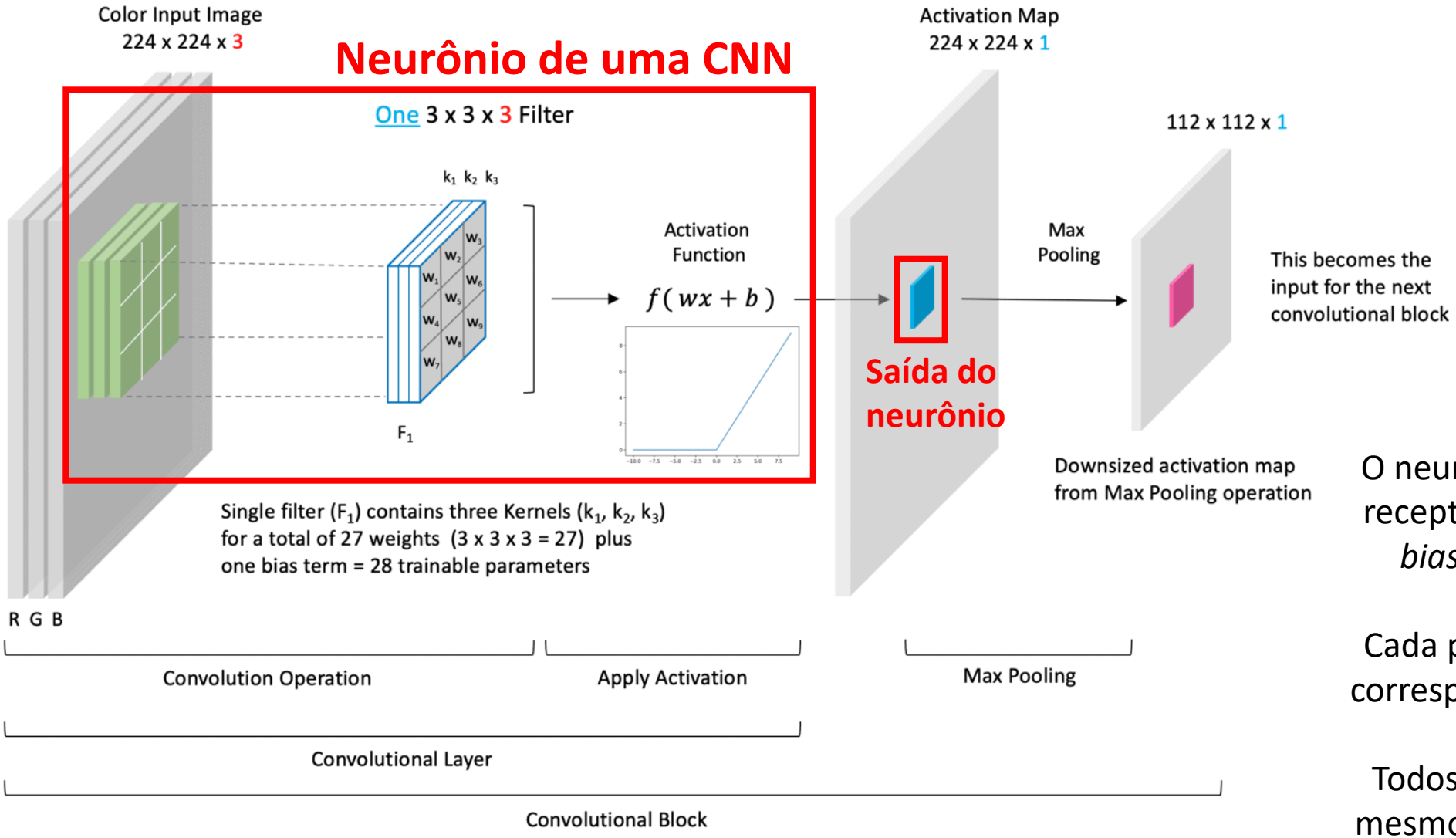


- Na figura, o *max pooling* **diminui a resolução** da imagem, **preserva as ativações mais fortes**, como as **bordas do cérebro**, e **descarta os detalhes de baixa intensidade e a posição exata** dentro de cada janela de *pooling*.

Bloco de convolução



O neurônio de uma CNN

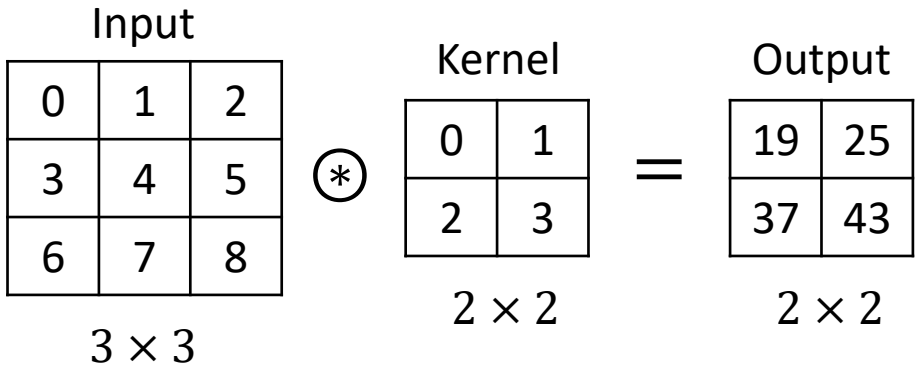


O neurônio é composto pelo campo receptivo, pelo *kernel*, pelo peso de *bias* e pela função de ativação.

Cada posição do mapa de ativações corresponde à saída de um neurônio.

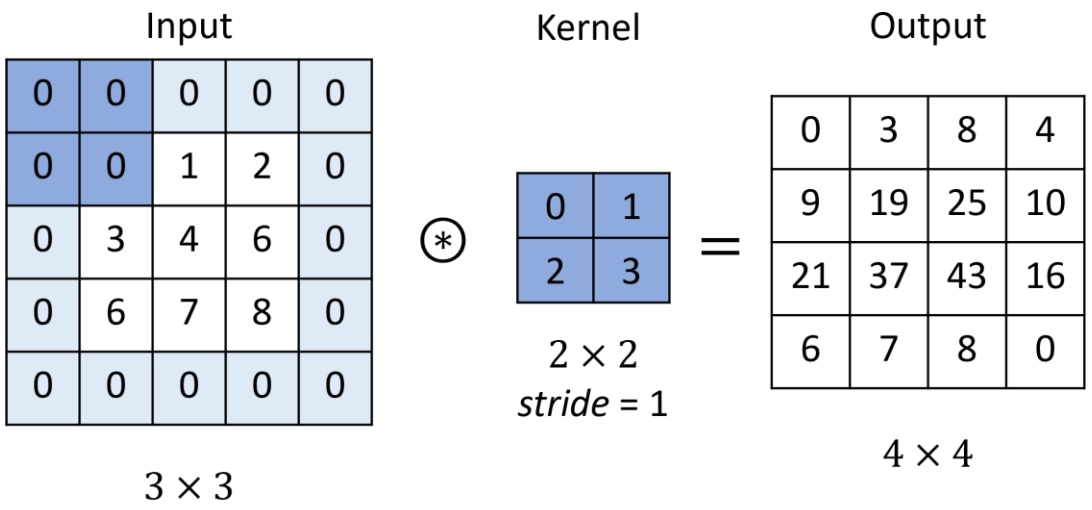
Todos os neurônios que geram um mesmo mapa usam o mesmo *kernel*.

Padding ou preenchimento



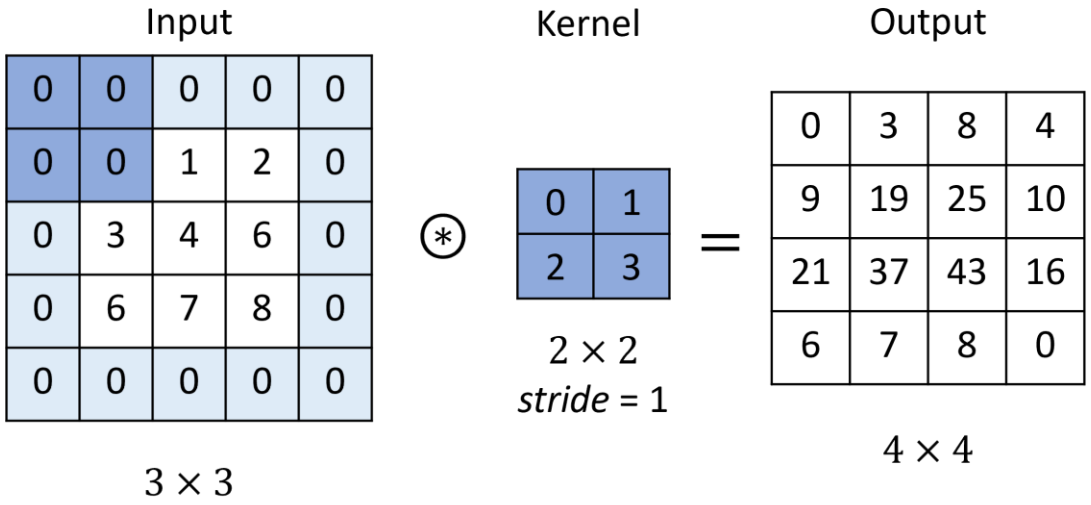
- **Após a aplicação de muitas convoluções** sucessivas, as **imagens tendem** a se **tornar** consideravelmente **menores** do que as da entrada.
- Por exemplo, se temos uma imagem de entrada com 240×240 *pixels*, após dez camadas de convolução 5×5 com *stride* igual a 1, ela é reduzida para 200×200 *pixels*.
- Isso reduz a imagem em $\approx$17%.

Padding ou preenchimento



- A convolução **reduz o tamanho do mapa de características** e faz com que os *pixels* das **bordas participem de menos convoluções que os centrais** (subutilização), podendo perder informações nas bordas.
- **O *padding* adiciona *pixels* de preenchimento** às **bordas da entrada**, **controlando** assim o **tamanho dos mapas de características de saída** de uma camada convolucional.

Padding ou preenchimento

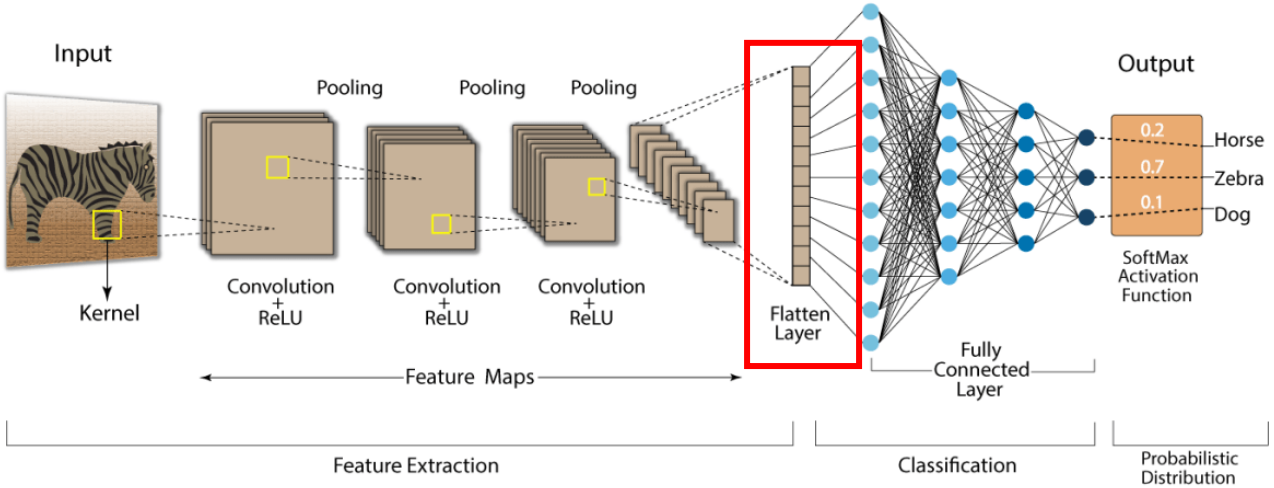


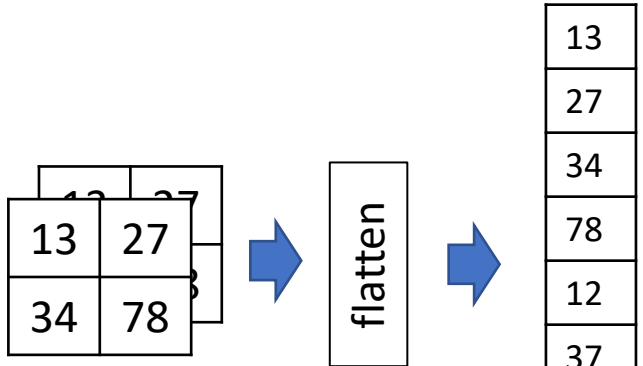
$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{out} : dimensão da saída
n_{in} : dimensão da entrada
k : dimensão do kernel
p : quantidade camadas de *padding*
s : tamanho do *stride*
$\lfloor x \rfloor$: função piso retorna o maior inteiro menor ou igual a x.

- Em geral, os ***pixels* de *padding* são feitos iguais a zero**.
- O *padding* garante que todos os *pixels* **contribuam de forma mais igualitária** para a extração de características.
- A equação ao lado dá o **tamanho da saída de uma camada convolucional** ao longo de **uma dimensão** (altura ou largura).

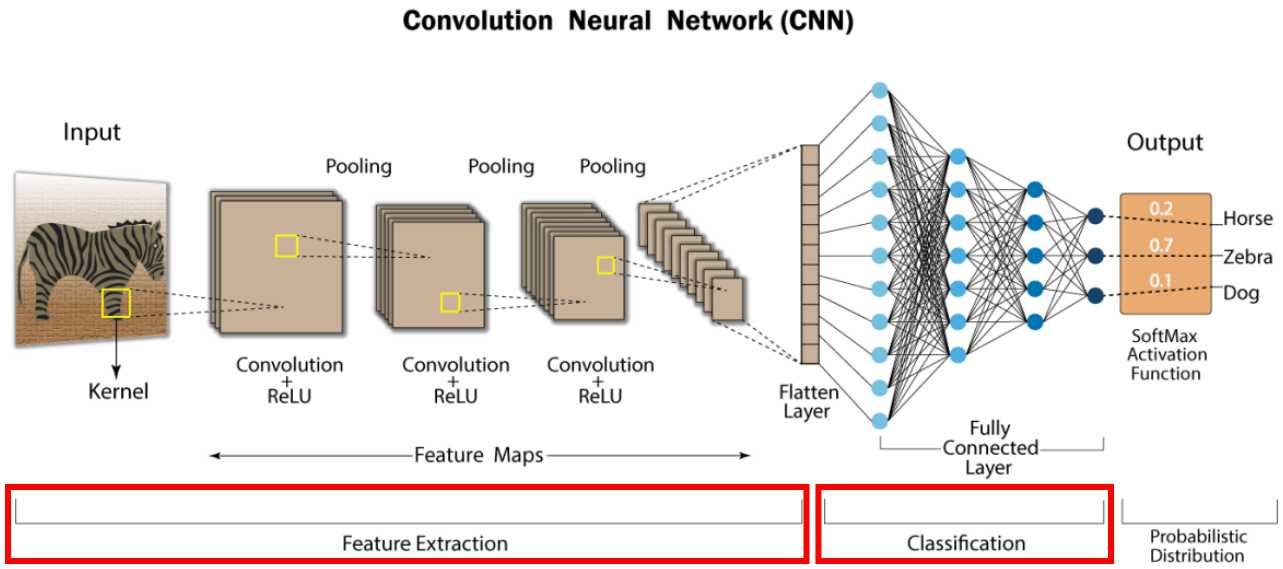
Camada de *flatten*





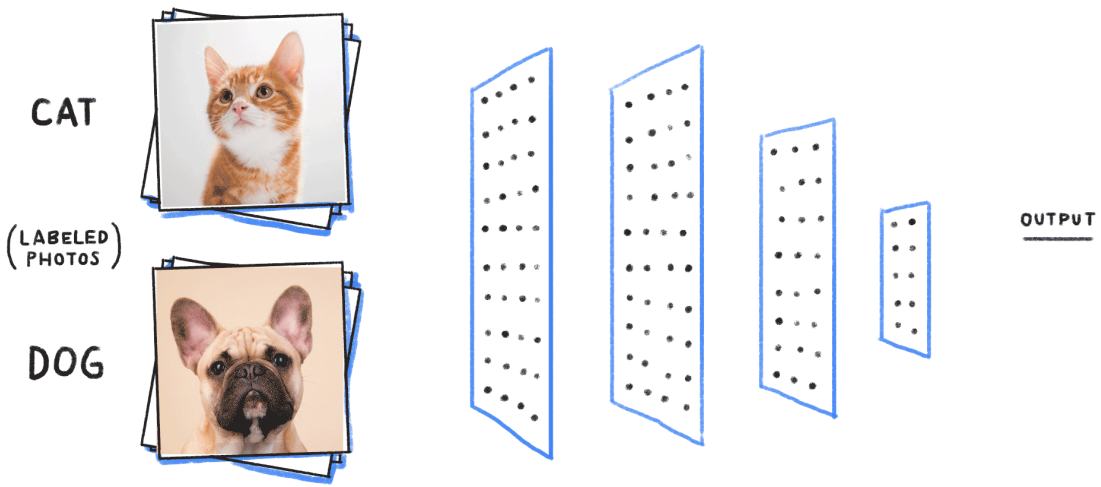
- Na saída da **etapa de extração de características** temos **vários mapas de características**, formando um **volume 3D** de dados.
- Porém, as **camadas densas** esperam **vetores 1D como entrada**.
- Assim, a camada de achatamento transforma os mapas de características em vetores 1D.
- Essa camada não possui parâmetros treináveis.

Camadas densas



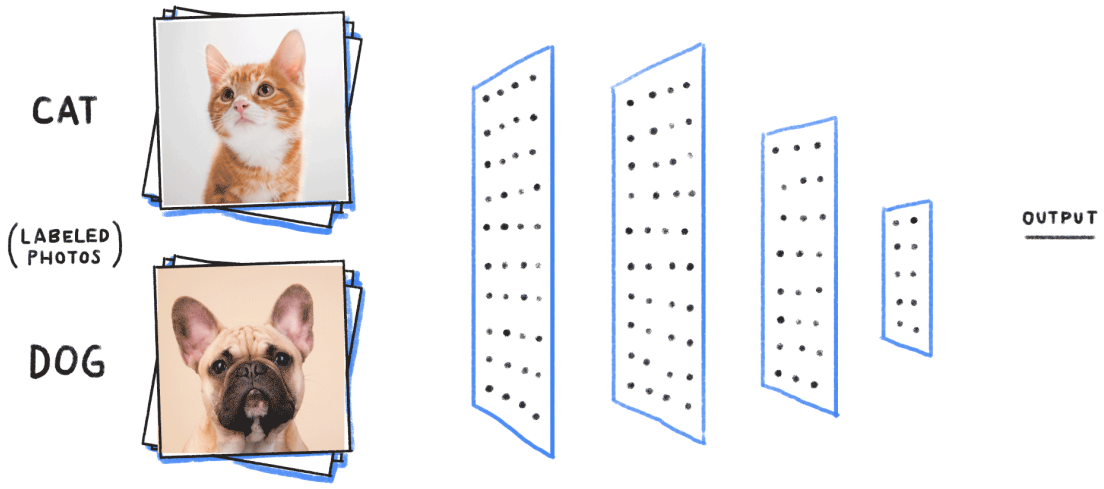
- Depois das camadas convolucionais e de achatamento, as **CNNs têm uma ou mais camadas densas**.
- As camadas densas
  - recebem o vetor 1D com **todas as características extraídas** pela rede,
  - **combinam todas as características extraídas**,
  - geram **pontuações** para cada classe
  - e as **normalizam**, usando a função *softmax*, gerando uma predição final (probabilidades).

Camadas densas



- Em outras palavras, elas **usam as características detectadas para decidir a classe**.
- Por exemplo, se as características de um gato são ativadas, a rede dá uma alta probabilidade para a classe gato.
- Por que usar camadas densas no final?
  - Convoluções são locais (olham pedaços da imagem).
  - As camadas densas olham tudo ao mesmo tempo.
- Ou seja, as camadas densas aprendem **relações globais entre características**.

Camadas densas



- As funções de ativação das camadas densas, com exceção da última, em geral, são ReLU ou suas variantes.
- A camada densa de saída usa funções de ativação:
  - *Softmax*, para classificação multiclasses.
  - Logística, para classificação binária e multi-rótulo.
  - Identidade, para regressão.

Treinamento de CNNs

- Assim como outras redes neurais, as CNNs são treinadas **usando-se o algoritmo da retropropagação do erro**.
- O objetivo é **aprender os pesos dos filtros, os de *bias* e das camadas densas**.
- Em geral, usa-se o **gradiente descendente em *mini-batches*** para a atualização dos pesos, pois com ele, podemos variar a complexidade computacional.
- As funções de erro mais usadas são o EQM, para tarefas de regressão, e a entropia-cruzada, para tarefas de classificação.

Devido a sua alta capacidade, redes profundas (MLPs, CNNs, e outras) podem facilmente se **sobreajustar** aos dados de treinamento

Como podemos mitigar o sobreajuste em modelos profundos?

Como evitar o sobreajuste

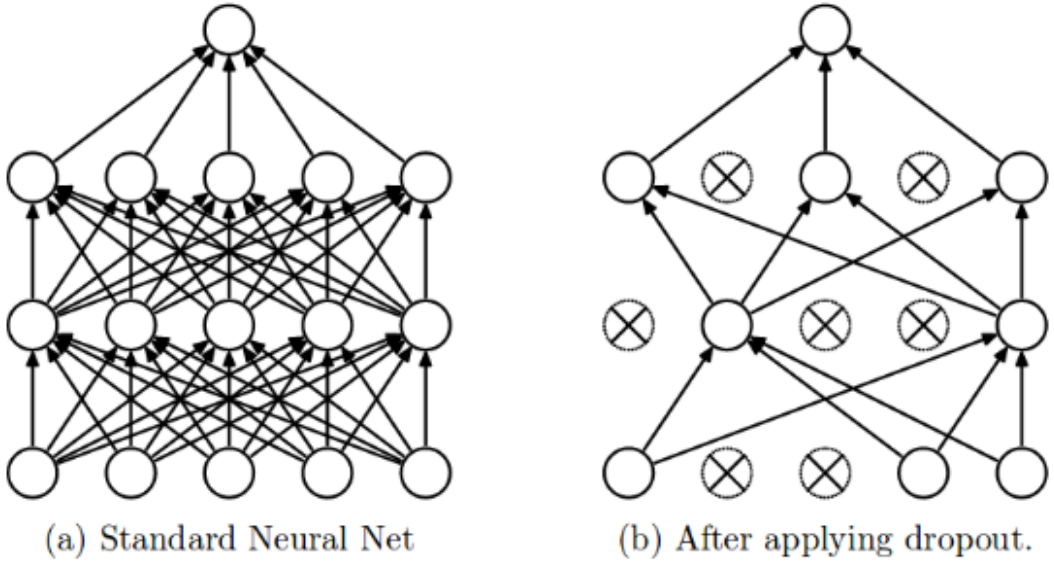
- Além das abordagens que já discutimos, como
  - **aumentar o *dataset***,
  - **reduzir a complexidade** (i.e., tamanho) do modelo
  - e **aplicar técnicas de regularização** como penalizações L1/L2 e *early-stopping*,

podemos usar três outras técnicas para evitar o *overfitting:*

- *dropout,*
- *data augmentation,*
- e *transfer learning* (Anexo I)

- Essas **técnicas se aplicam a qualquer tipo de rede neural, não apenas a CNNs**.

Dropout

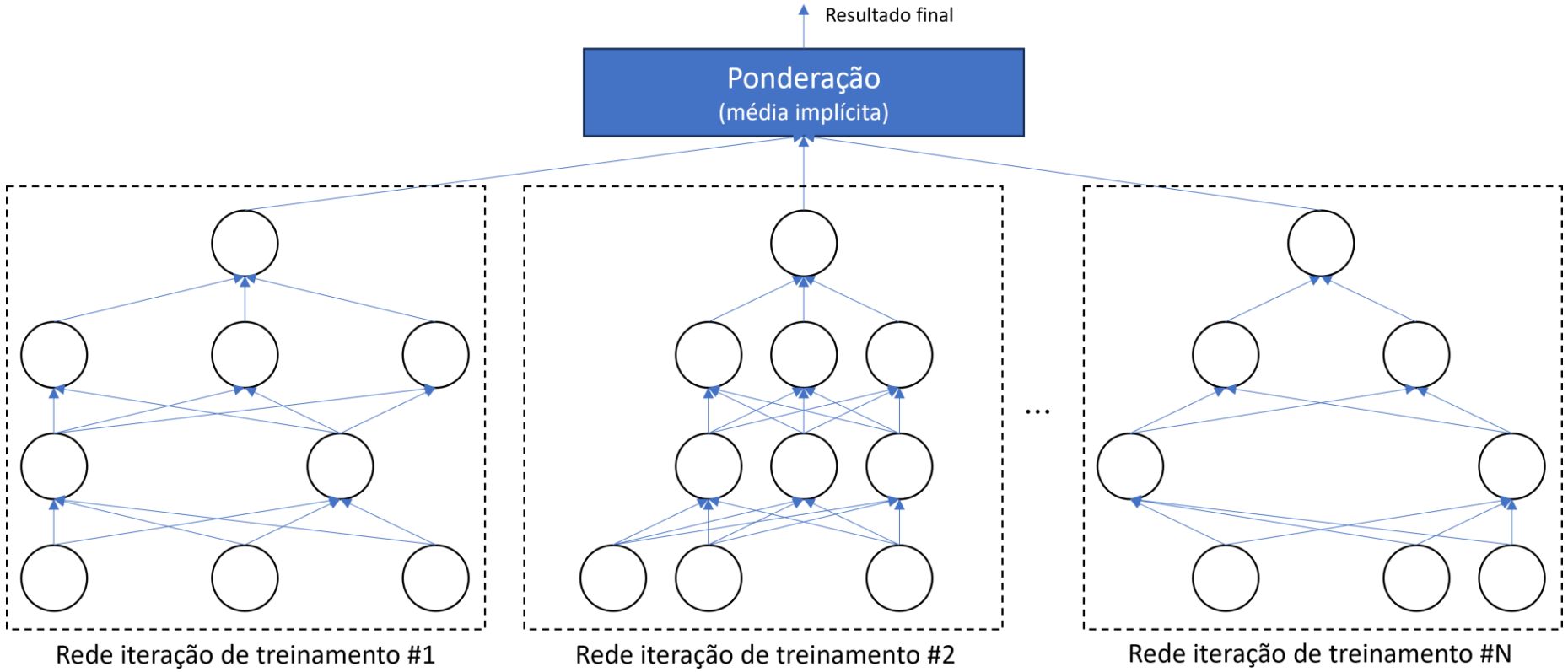


(a) Standard Neural Net

(b) After applying dropout.

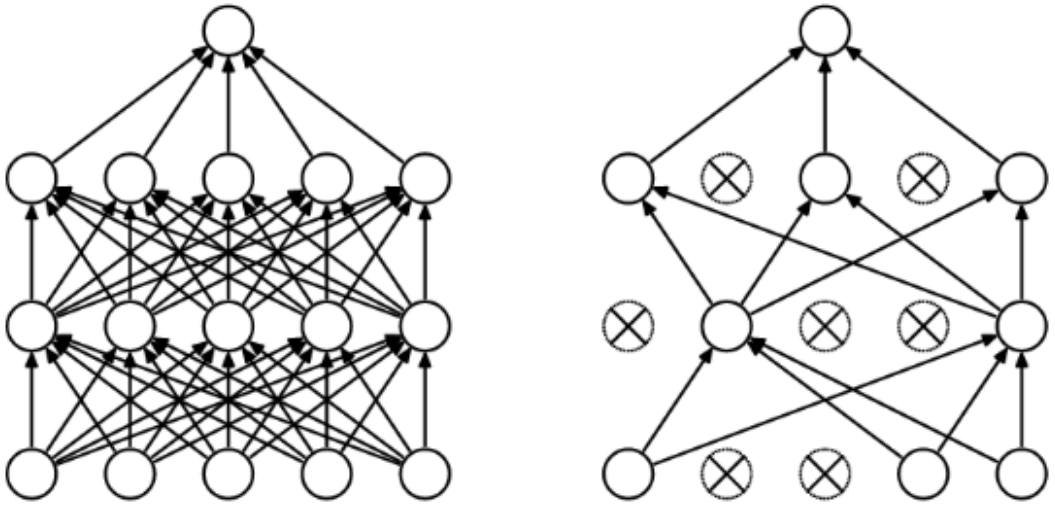
- O ***dropout*** é uma técnica de regularização.
- Envolve "**desligar**" **aleatoriamente** os nós das camadas **durante o treinamento**, cada um com probabilidade *p* (um hiperparâmetro).
- Os nós desligados **não contribuem para a saída** da **rede nem têm seus pesos atualizados** naquela iteração de treinamento.
- Assim, a **cada iteração treina-se uma sub-rede menor**, o que **reduz a dependência excessiva entre nós** e a chance de sobreajuste.

Dropout



- A cada **iteração de treinamento** o **subconjunto de nós desligados muda**, e por isso é **como se uma rede diferente fosse treinada a cada vez**.

Dropout

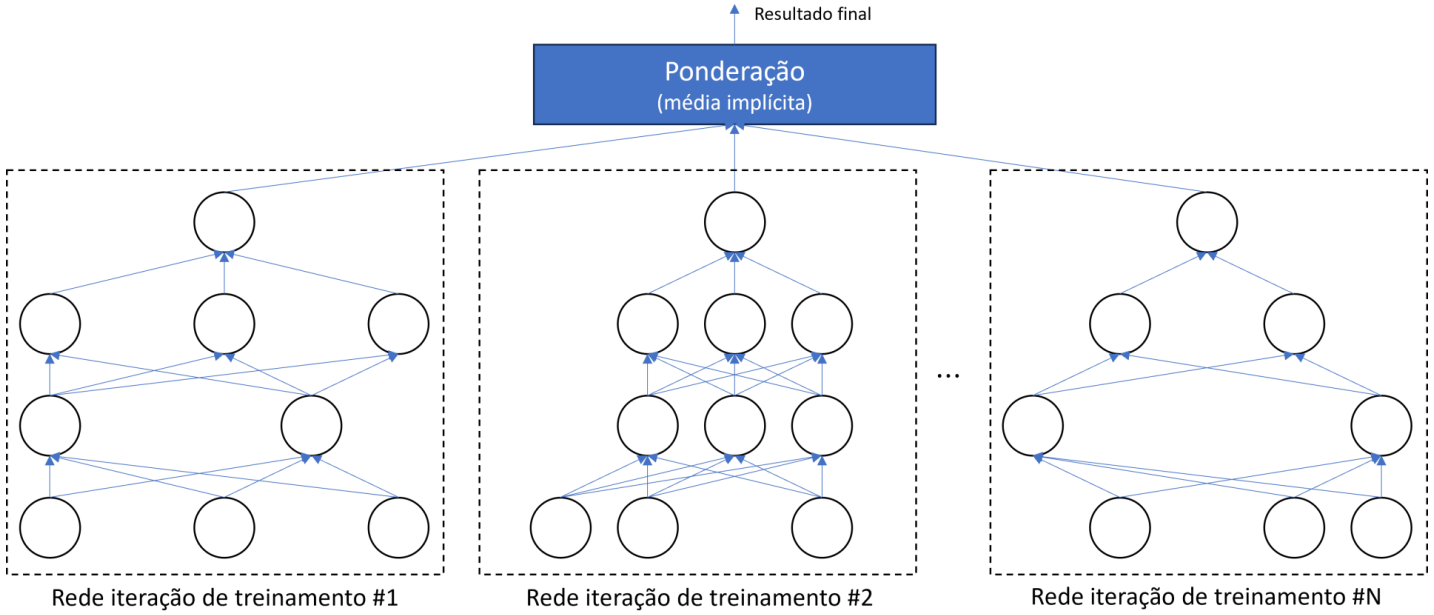


(a) Standard Neural Net

(b) After applying dropout.

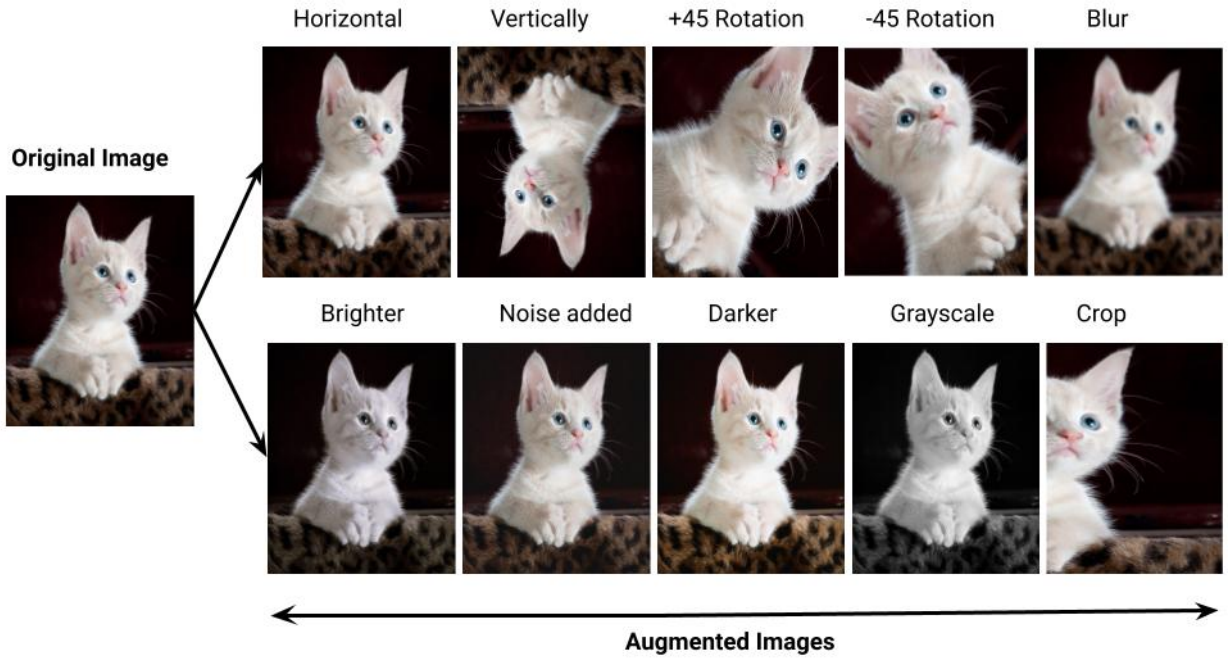
- Com o *dropout*:
  - os nós não conseguem depender uns dos outros de forma excessiva
  - cada nó é forçado a aprender características úteis por si só.
- Ou seja, o *dropout* força os nós a funcionarem bem mesmo se outros nós desaparecerem.
- Isso força a rede a **aprender representações mais generalizáveis**.

Dropout



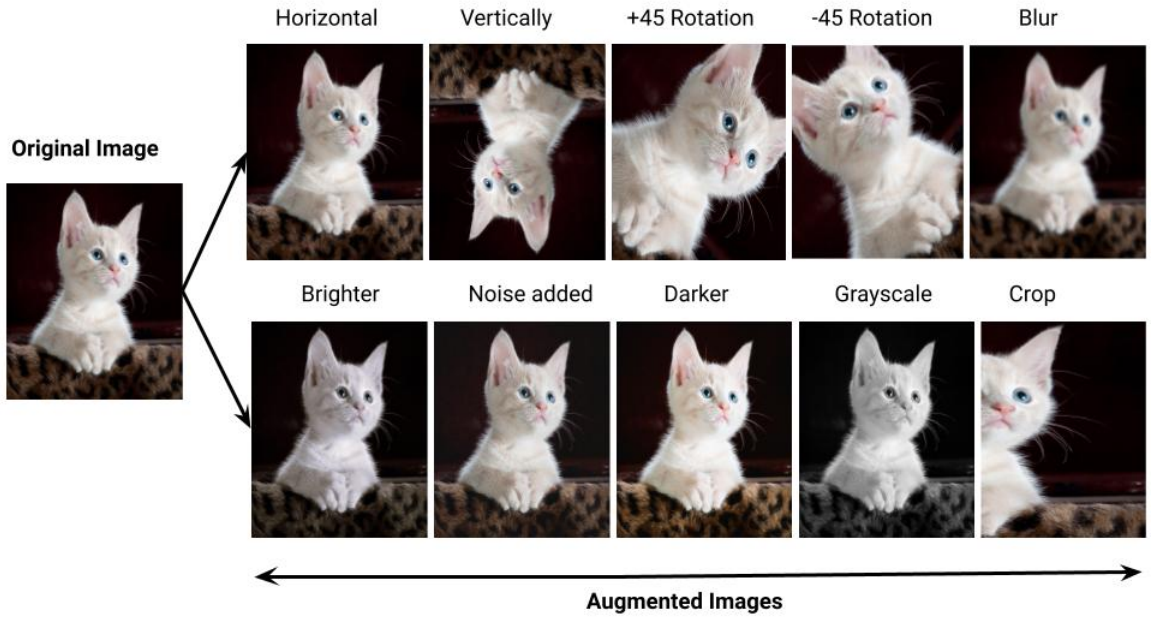
- Durante a **inferência**, o *dropout* não é aplicado (todos os nós são usados).
- Durante a **inferência**, a rede completa **aproxima a média das predições de muitas sub-redes** treinadas com *dropout*.
- Essa **combinação implícita reduz o efeito do sobreajuste**, fazendo com que a rede generalize melhor.

Data augmentation



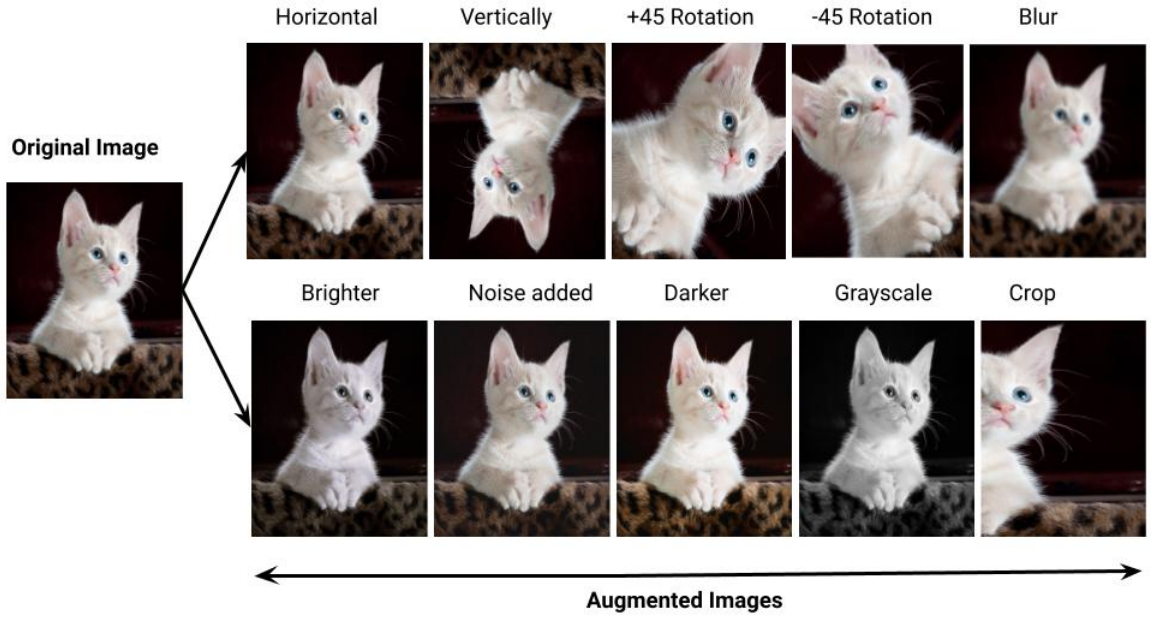
- **Podemos aumentar artificialmente os dados**.
- O objetivo é **aumentar** a **diversidade** dos dados de **treinamento criando novos exemplos a partir dos dados existentes**.
- Essa técnica ajuda a melhorar o desempenho do modelo, tornando-o mais **robusto** e capaz de **generalizar melhor**.

Data augmentation



- Quando o problema envolve **imagens**, podemos aplicar **transformações**, como
  - **rotação**,
  - **espelhamento**
  - **corte**,
  - modificação do **brilho**,
  - adição de **ruído**,
  - **desfoque**,
  - transformação para **escala de cinza**,
  - etc.
- A rede é treinada com a **mesma imagem, mas com diferentes transformações**.

Data augmentation



- O ***data augmentation*** **torna as CNNs robustas a variações de**
  - rotação,
  - qualidade,
  - luminosidade,
  - escala e posição dos objetos nas imagens.

das imagens.

- Assim, a rede **aprende a reconhecer padrões de forma menos sensível a esses fatores**, desde que as variações estejam representadas nas transformações usadas.

Exemplo

- Exemplo: cnn.ipynb



Lista de exercícios

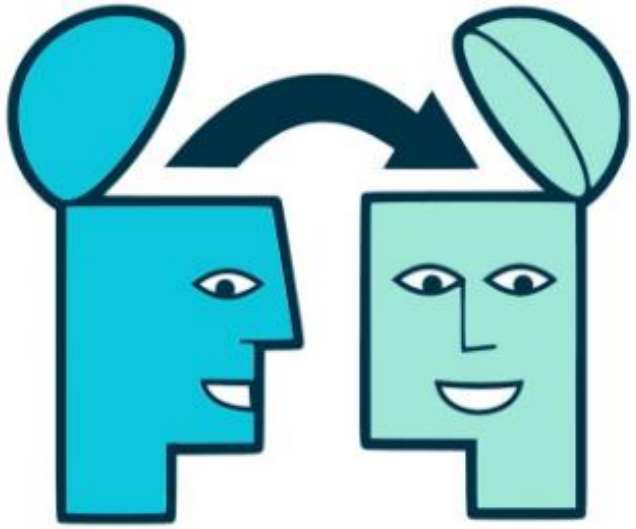
Lista #10 - CNN

Perguntas?

Obrigado!

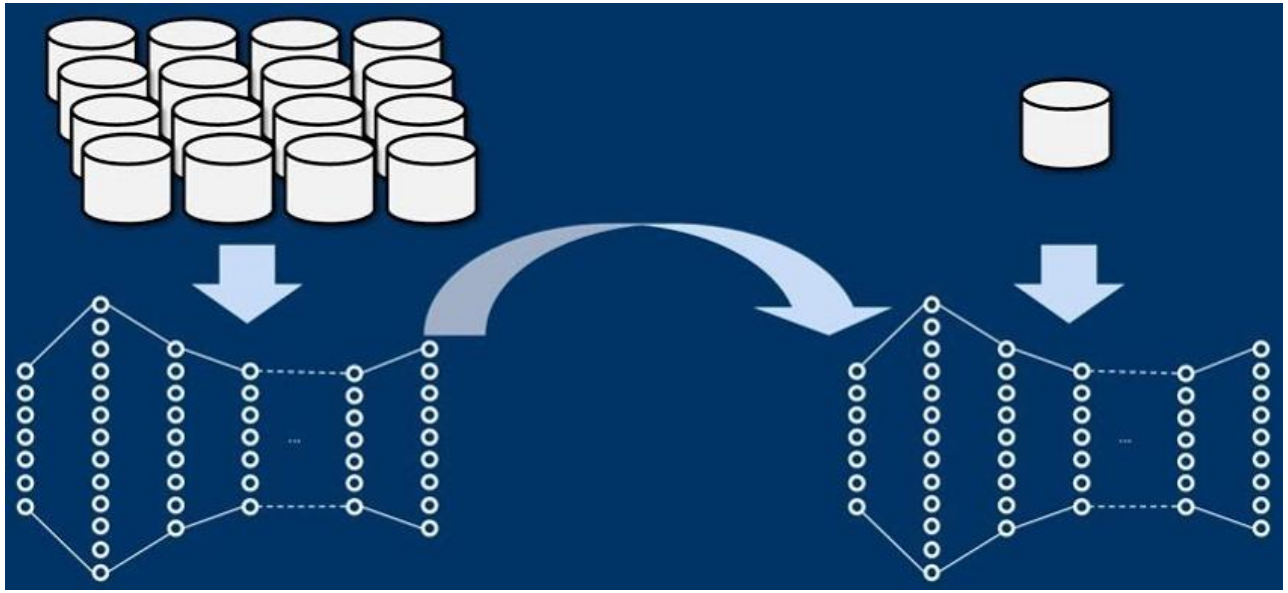
Anexo I:*Transfer Learning*

Transfer learning



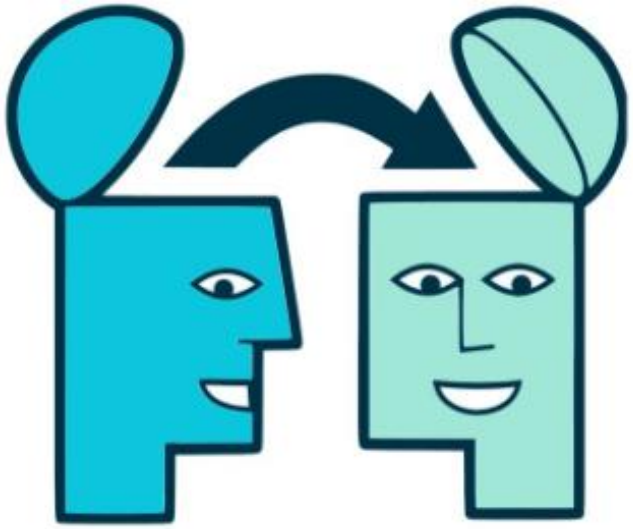
- ***Transfer learning*** ou **transferência de aprendizado** é uma abordagem que envolve o uso de um **modelo pré-treinado em uma tarefa** (ou **domínio**) relacionada **como ponto de partida para uma nova tarefa**.
- Por exemplo, usar um **modelo pré-treinado em um grande conjunto de dados com diferentes tipos de plantas** e **ajustá-lo** para a **classificação de doenças em pés de feijão**.

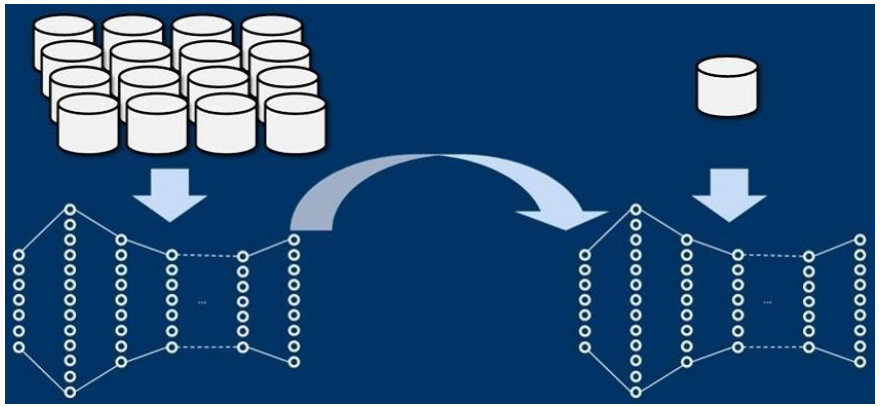
Transfer learning



- Usamos o **conhecimento adquirido** por um **modelo treinado em uma grande base de dados** como **ponto de partida** para treinar um modelo com uma **base de treinamento, em geral, menor**, **mas que seja relacionada à primeira**.

Transfer learning



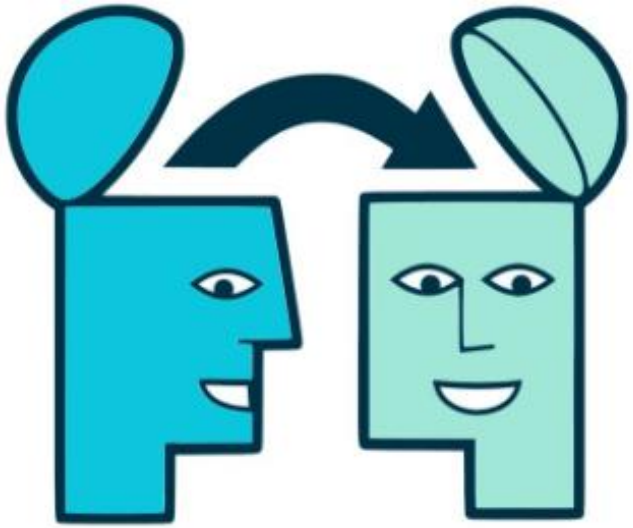


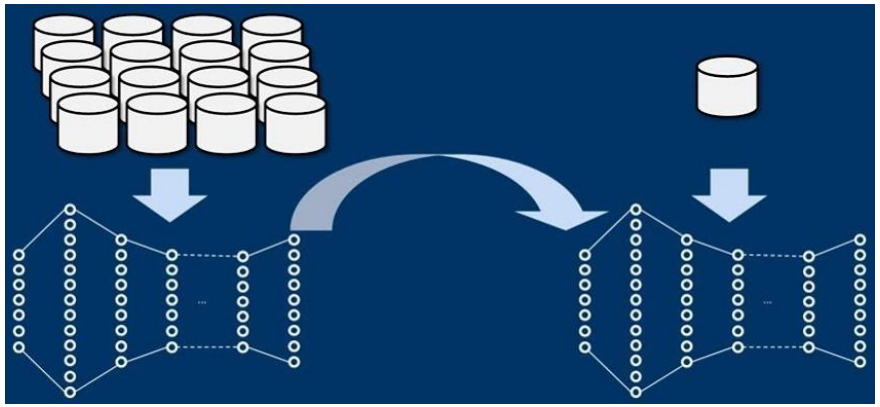
- Suponham que tenhamos uma **rede pré-treinada** para classificar imagens de **100 classes diferentes**, incluindo **animais**, **plantas**, **veículos**, etc. e queremos treinar uma **nova rede para classificar tipos específicos de veículos**.
- Por essas tarefas serem semelhantes, **podemos reutilizar partes da primeira rede**.
  - Reutilizamos as camadas iniciais, que detectam características genéricas.
  - Substituímos a camada final (classificadora) por uma nova, com saídas para as classes de veículos, e a treinamos com os novos dados.

Transfer learning

- Modelos pré-treinados em grandes conjuntos de dados aprendem a **extrair características relevantes dos dados**.
- Os **pesos que detectam essas características são transferidos** para o novo modelo, que se beneficia de uma **inicialização mais informada**.
- As camadas iniciais dos modelos pré-treinados detectam padrões básicos (bordas, cores, texturas).
- As intermediárias detectam formas e partes de objetos.
- As finais capturam características específicas da tarefa original, e por isso **reaproveitamos principalmente as iniciais**.

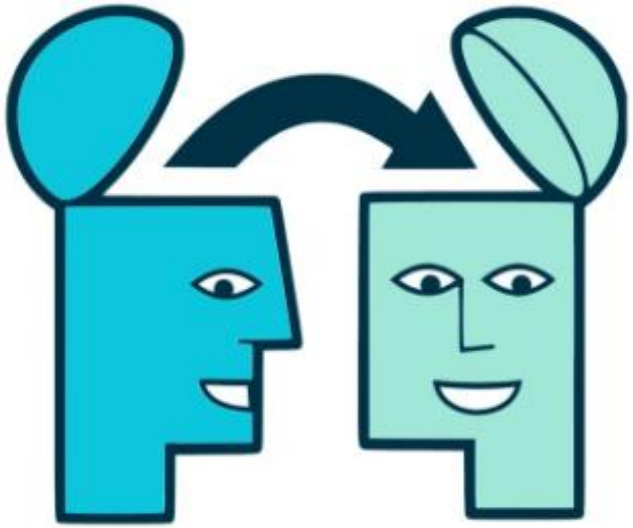
Transfer learning

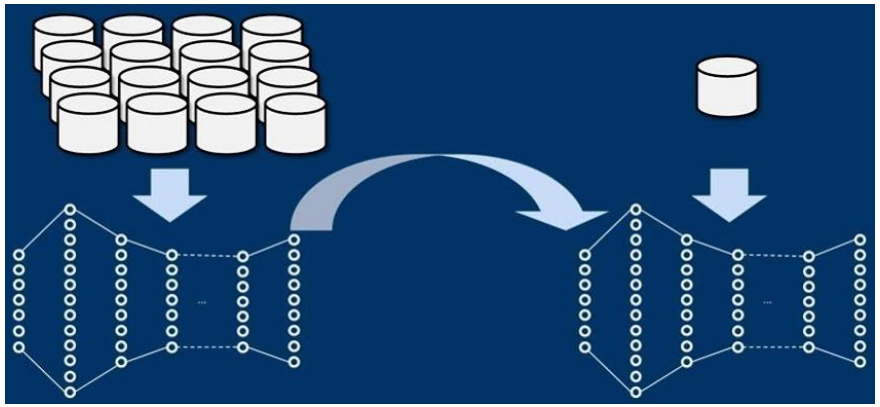




- O *transfer learning* **pode reduzir o risco de sobreajuste**, pois o novo modelo **já começa com características relevantes**, em vez de pesos aleatórios.
- Como o **modelo pré-treinado já adquiriu um bom entendimento de características genéricas** em uma tarefa relacionada, o **treinamento do novo modelo pode exigir menos dados** para alcançar um bom desempenho.

Transfer learning

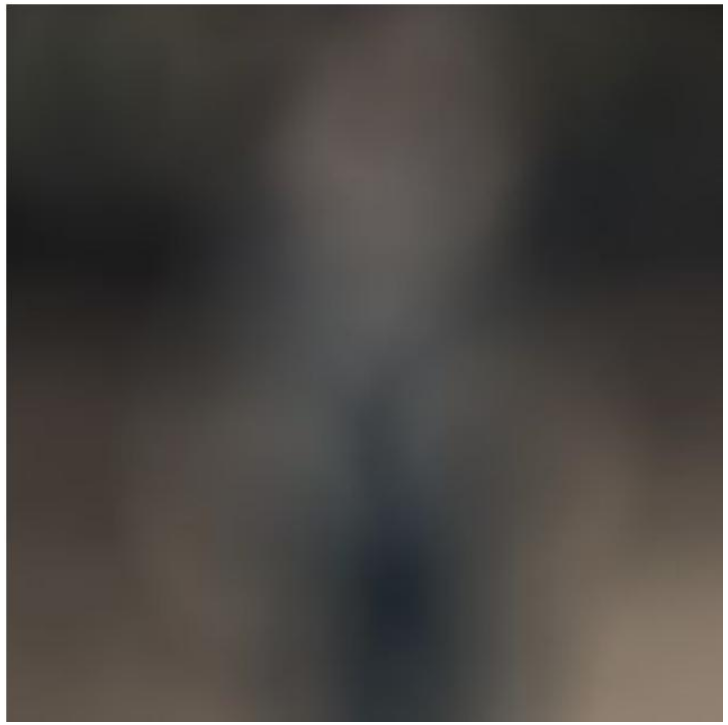




- O *transfer learning* é particularmente **útil quando os dados de treinamento são escassos**, o que é uma situação **propensa ao sobreajuste**.
- Por partir de pesos já treinados e exigir menos dados, o *transfer learning*:
  - acelera a convergência do treinamento;
  - requer menor poder computacional (o custo do pré-treinamento já foi pago por quem treinou o modelo original).

Anexo II: Exemplo de classificação de imagens

Exemplo de classificação de imagens



- Para entender melhor o que uma camada de convolução faz, vamos considerar um exemplo simples.
- Vamos supor que **não conhecemos o conteúdo da imagem** ao lado.
- Isso foi simulado deixando-a embaçada.

Exemplo de classificação de imagens



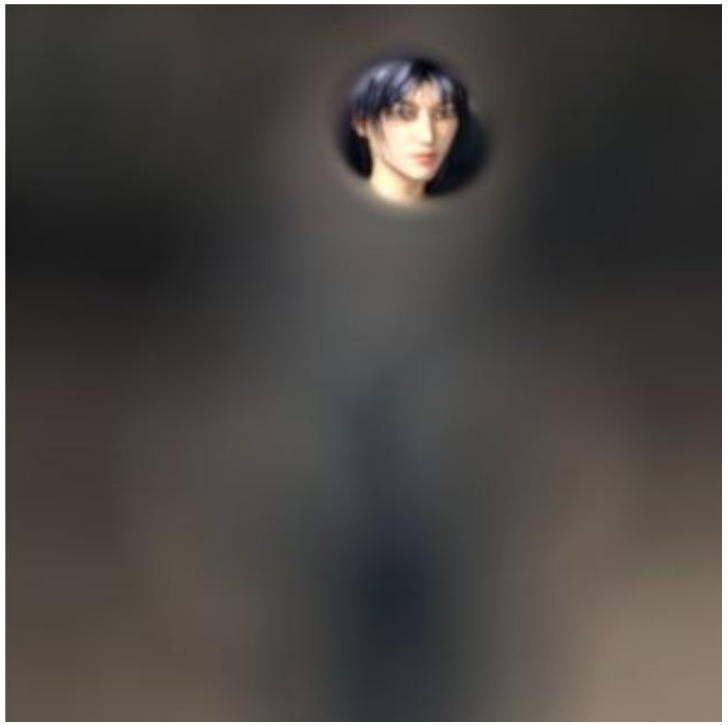
- Então, digamos que haja um **filtro ou um conjunto de filtros** que ao serem passados sobre a imagem **extraem as características** mostradas ao lado.
- Como podemos ver, **são duas formas verticais**, que se parecem com pernas humanas.

Exemplo de classificação de imagens



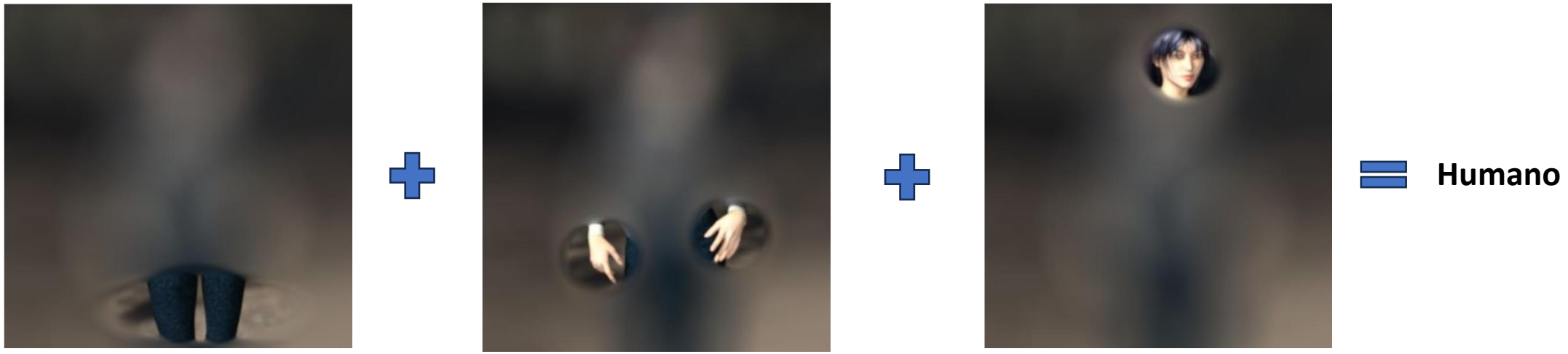
- Na sequência, **outros filtros extraem as características** mostradas ao lado.
- São pequenas formas cilíndricas que se projetam a partir de um cilindro maior.
- Nosso **cérebro vê isso** e **instantaneamente classifica como mãos**.
- Porém uma rede neural em treinamento ainda não sabe disso.
- Ela apenas sabe que um filtro pode extrair essas características.

Exemplo de classificação de imagens



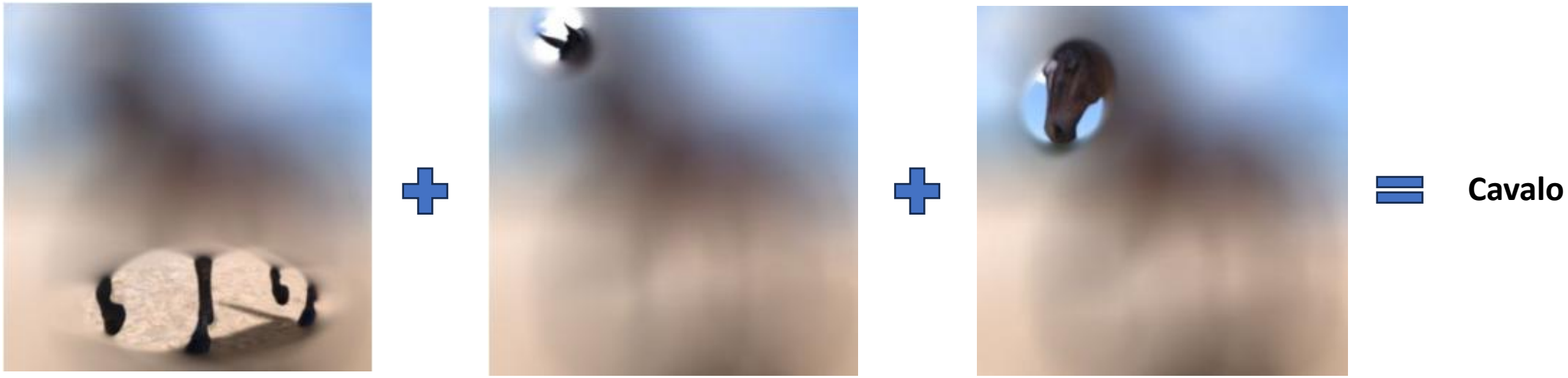
- Em seguida, **outros filtros extraem um círculo com outros dois círculos menores, uma saliência e algumas formas paralelas**.
- Nosso cérebro reconhece esse conjunto de pixels como um rosto com olhos, boca e nariz.
- Mas, novamente, o modelo não tem contexto para decidir o que esses pixels são.
- Ele só sabe que um filtro específico pode extrair essas características.

Exemplo de classificação de imagens



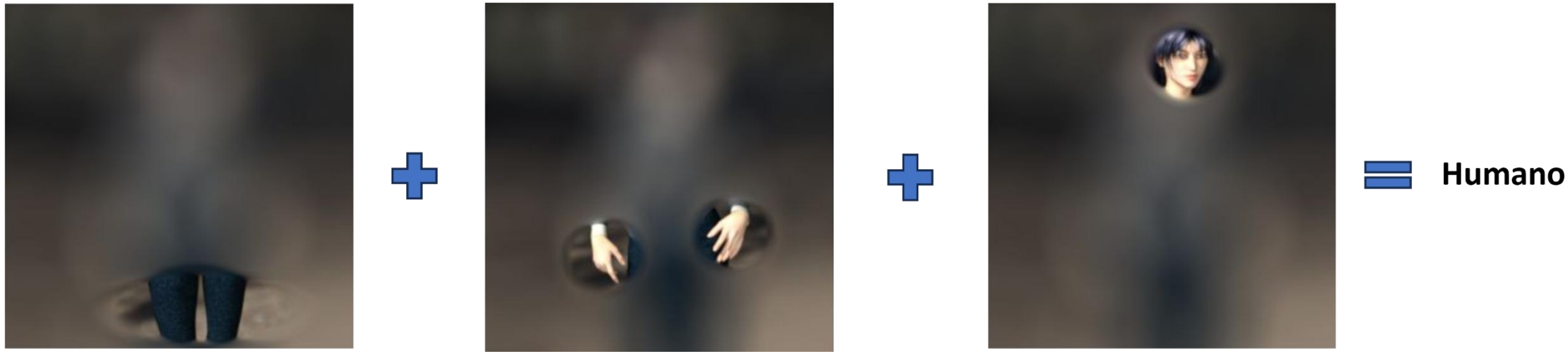
- Quando essas **características estão presentes em uma imagem rotulada como humana**, a rede neural pode ser **treinada para aprender filtros que extraem essas informações**.
- A rede **pode combinar as características extraídas pelos três filtros** (i.e., pernas, mãos e face) para detectar seres humanos em imagens inéditas.
  - A(s) camada(s) densa(s) faz(em) essa combinação.

Exemplo de classificação de imagens



- A **mesma rede pode ser treinada para identificar** diferentes **características** presentes nas imagens de **cavalos**.
- Assim, ao **aprender conjuntos de filtros que podem detectar humanos ou cavalos**, temos um **modelo de visão computacional** que pode **lidar** com **imagens complexas** e **predizer o que há nelas**.

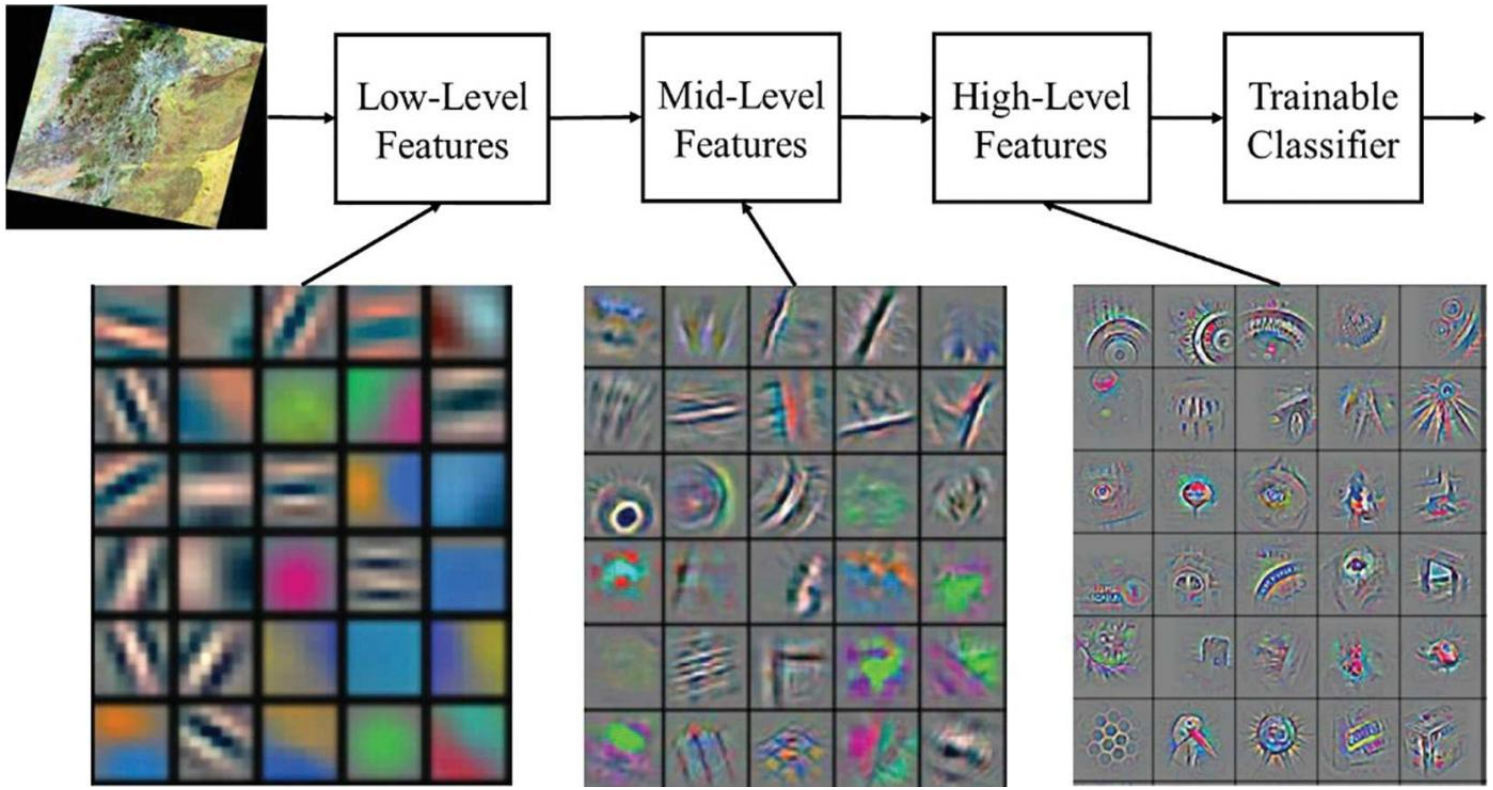
Observação



- Notem que usamos pernas, mãos e rostos como exemplos de características extraídas por um filtro.
- Essas são formas reconhecíveis pelo nosso cérebro, então as utilizamos para ilustrar o conceito.
- Porém, as redes convolucionais, muito provavelmente **aprenderão características imperceptíveis** a nós humanos.

Anexo III: Links interessantes

Características imperceptíveis

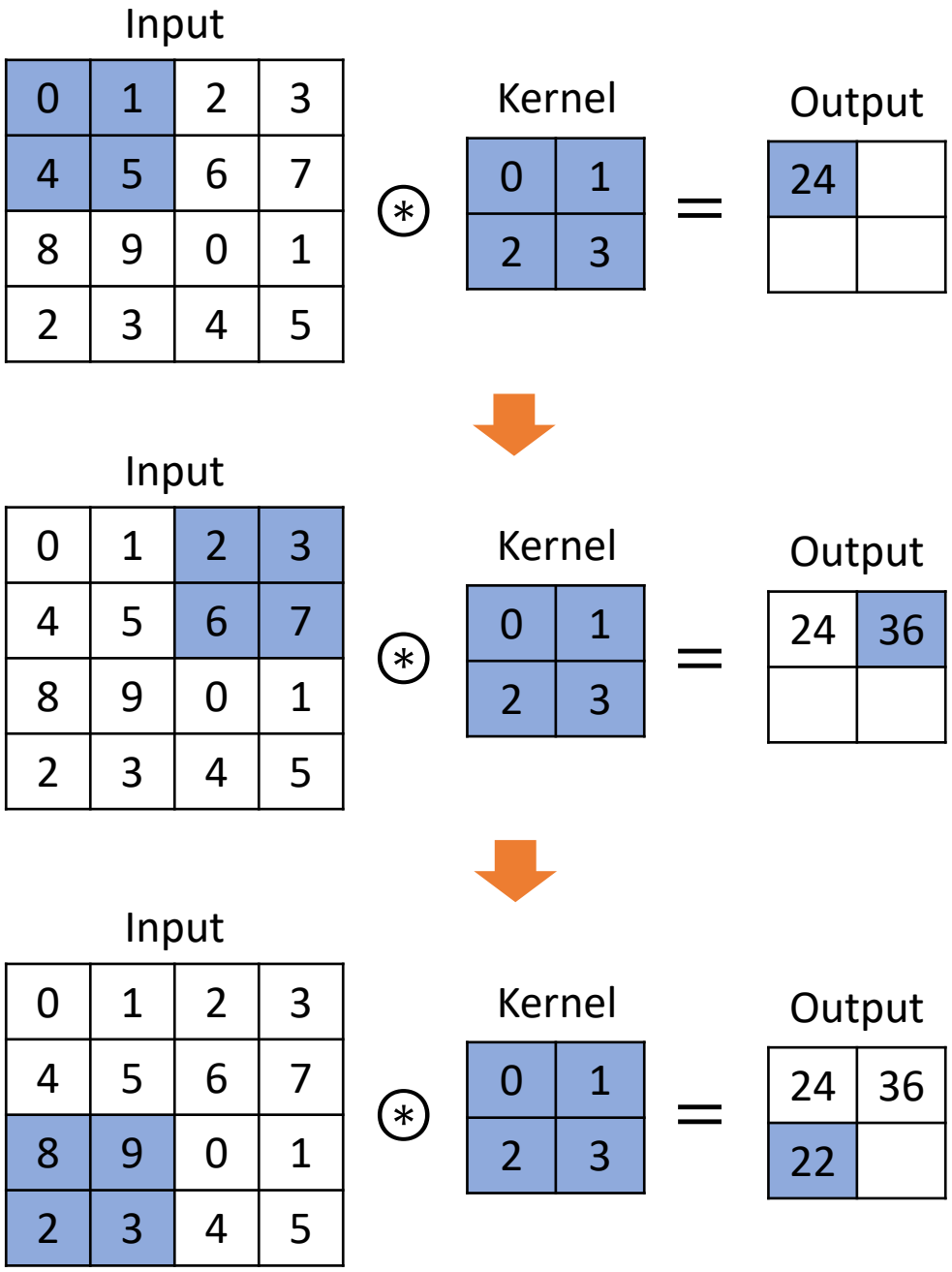


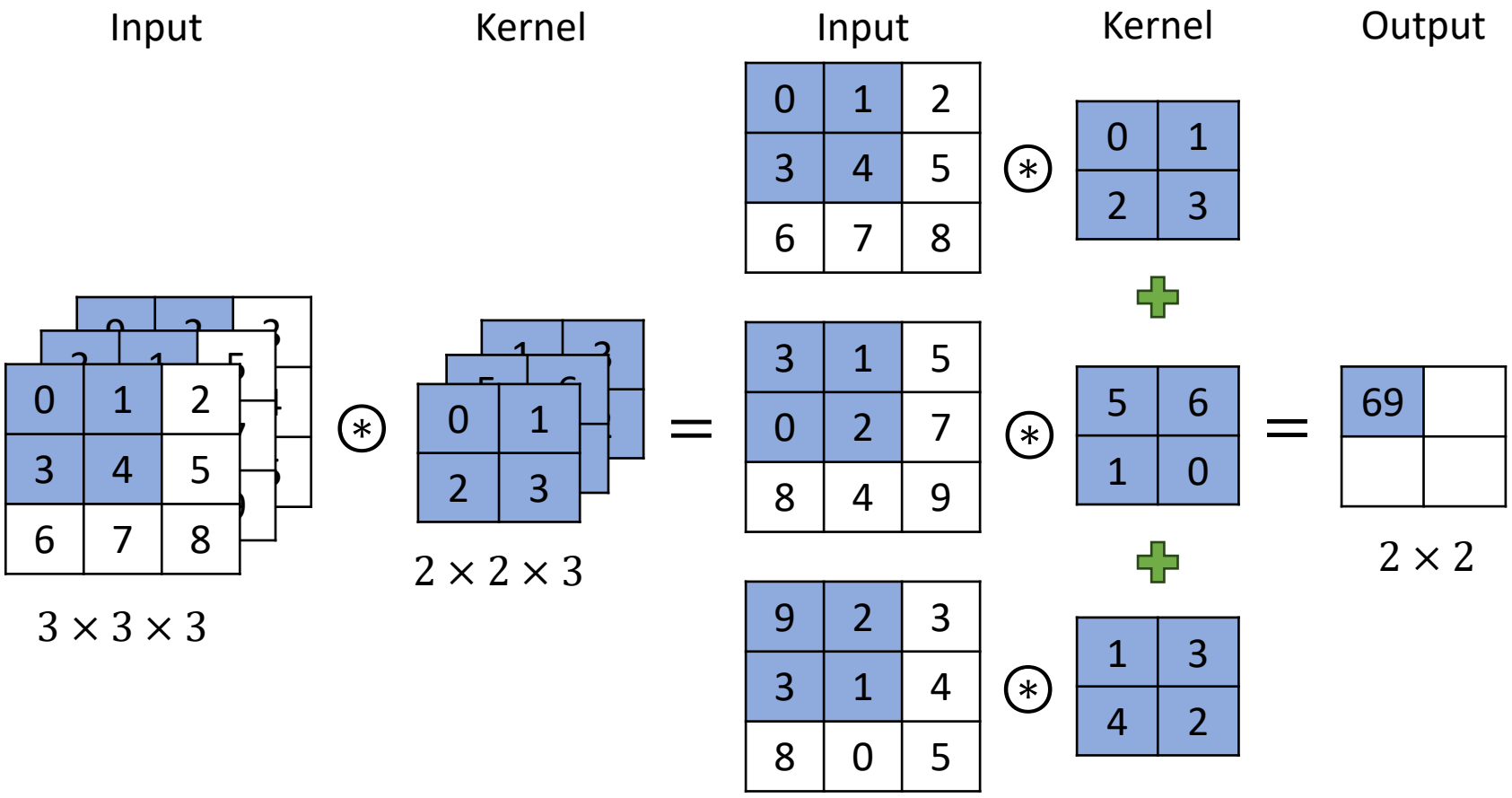
- No entanto, **quando filtros são treinados** para detectar características em imagens, **eles podem identificar coisas que são imperceptíveis para os seres humanos**.
- Podem existir padrões de *pixels* que correspondam a uma imagem rotulada, mas que não tenham significado aparente para nós.

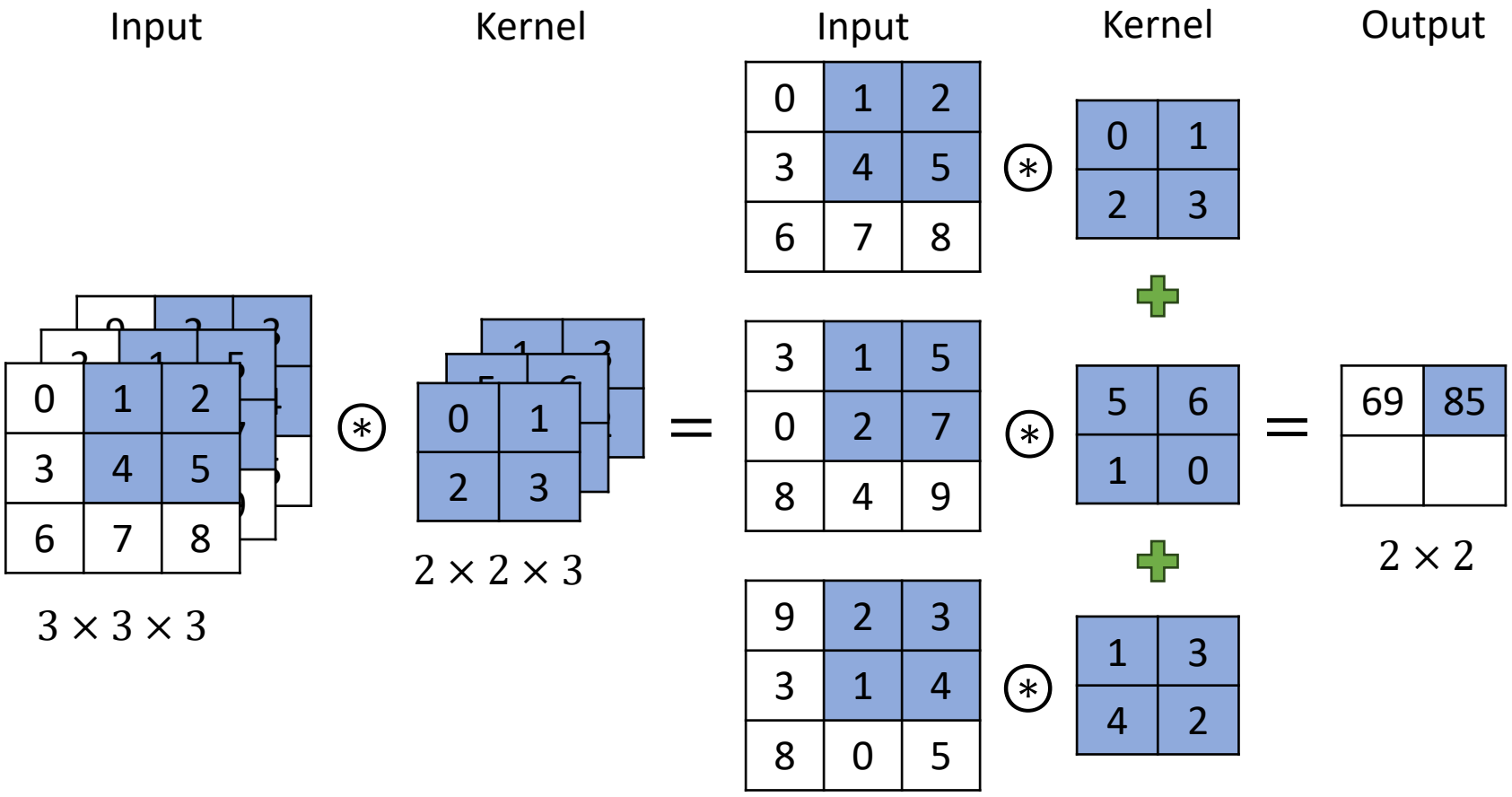
Explorando CNNs

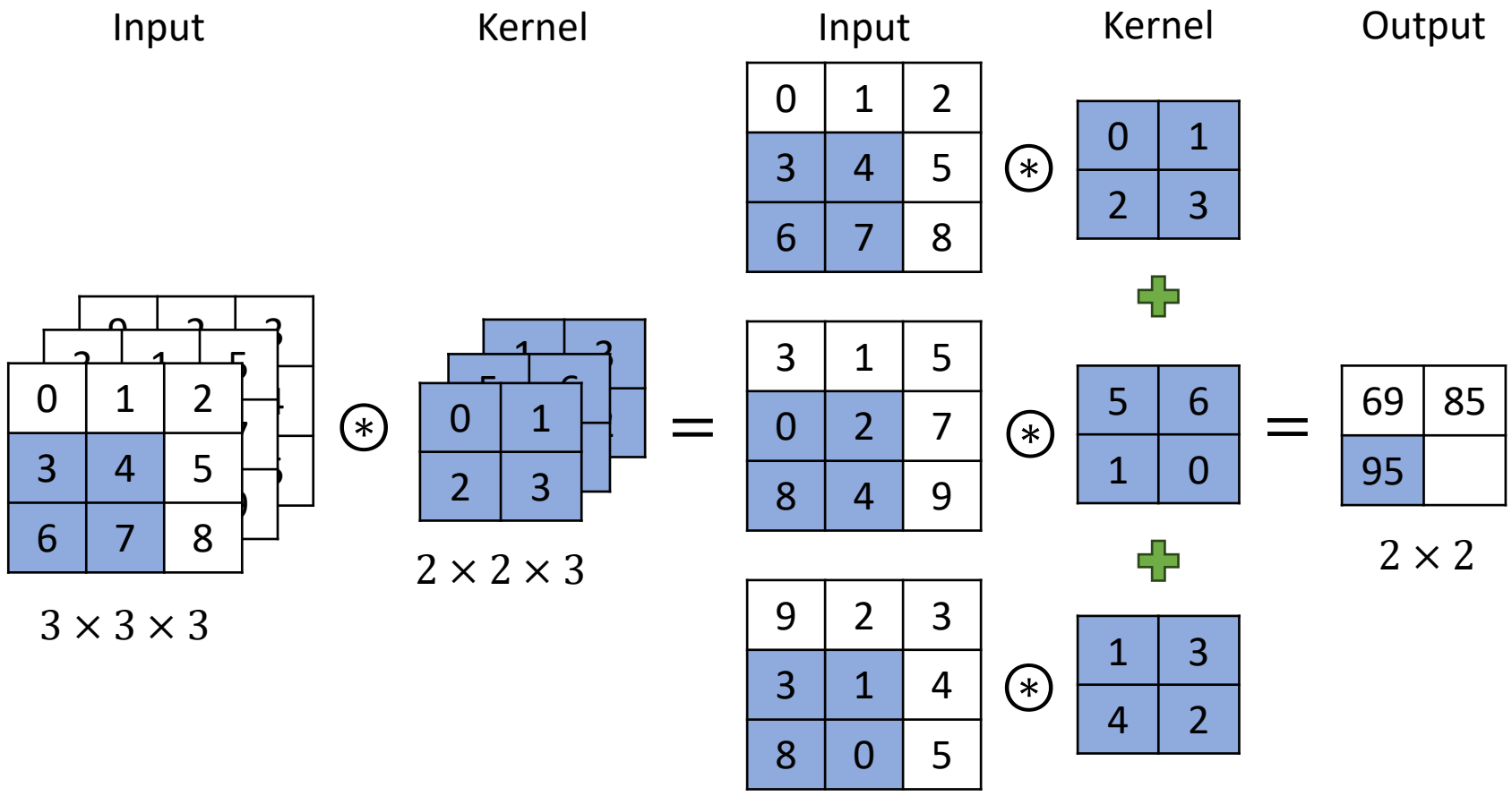
- CNN Explainer
  - https://poloclub.github.io/cnn-explainer/
- ConvNetJS MNIST demo
  - https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html
- ConvNetJS CIFAR-10 demo
  - https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html

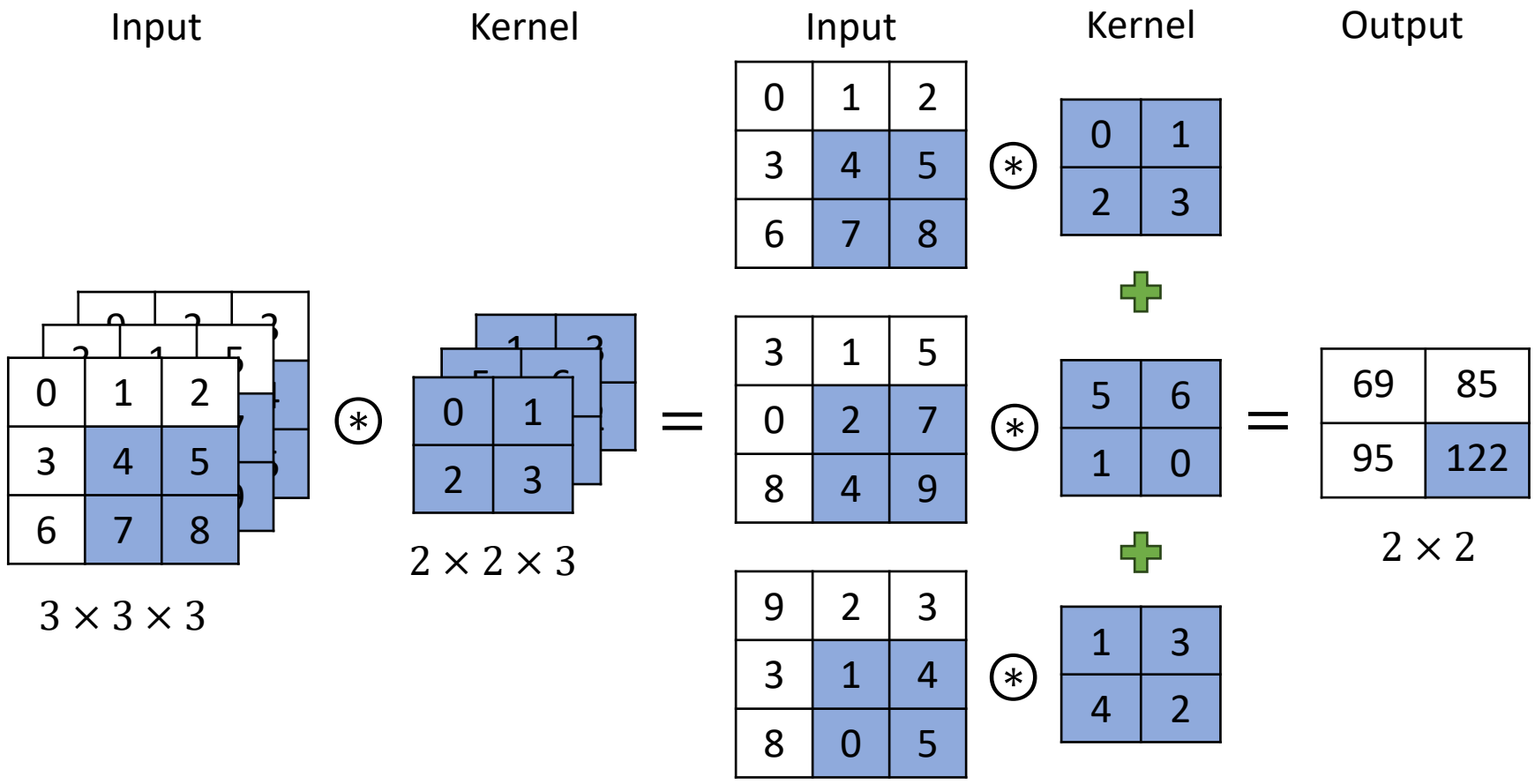
Figuras

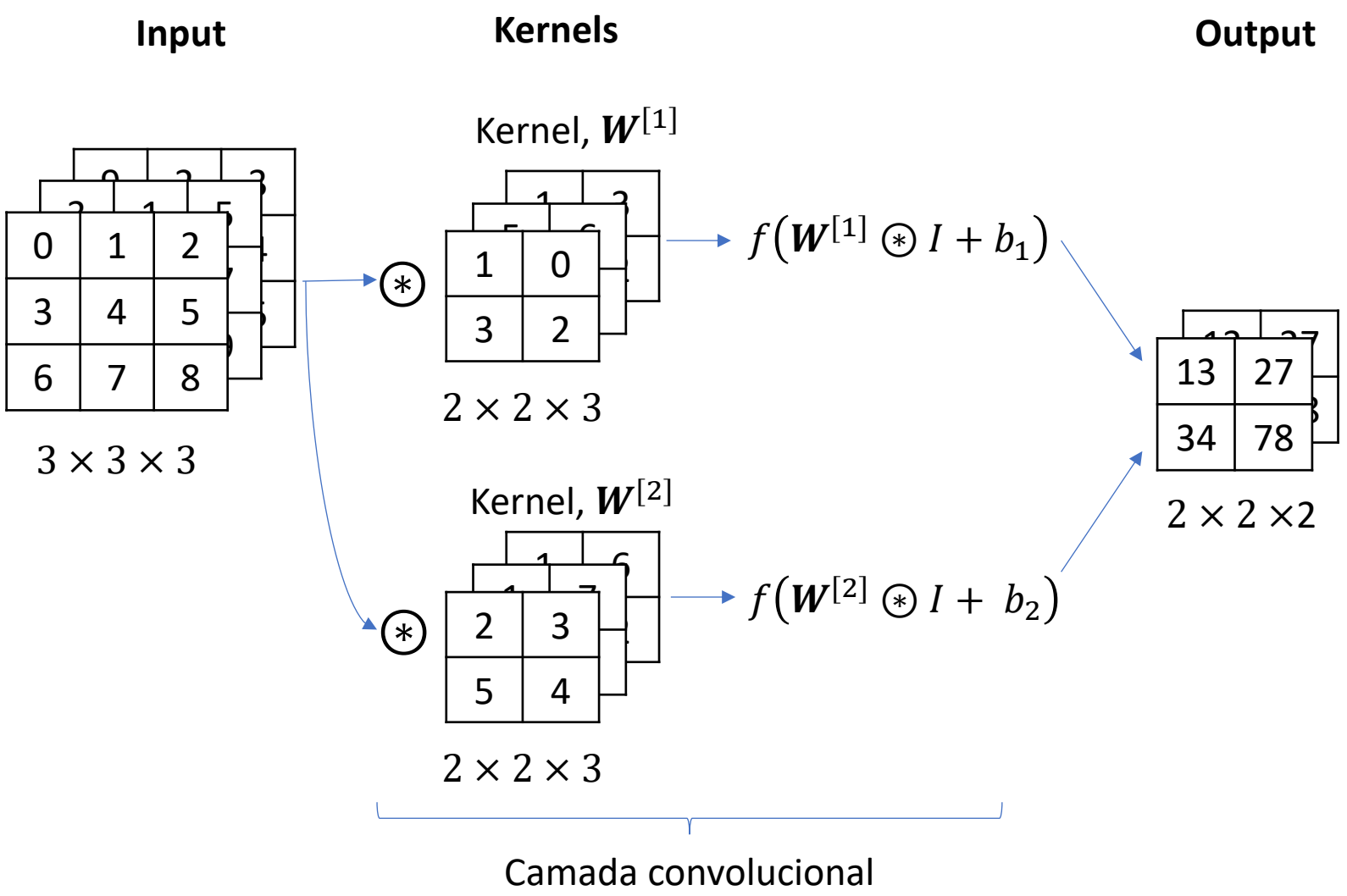

Input
0 1 2 3
4 5 6 7
8 9 0 1
2 3 4 5
⊛
Kernel
0 1
2 3
=
Output
24
Input
0 1 2 3
4 5 6 7
8 9 0 1
2 3 4 5
⊛
Kernel
0 1
2 3
=
Output
24 36
Input
0 1 2 3
4 5 6 7
8 9 0 1
2 3 4 5
⊛
Kernel
0 1
2 3
=
Output
24 36
22


Input
Kernel
Input
Kernel
Output
0 1 2
3 4 5
6 7 8
3 × 3 × 3
0 1
2 3
2 × 2 × 3
3 1 5
0 2 7
8 4 9
9 2 3
3 1 4
8 0 5
0 1
2 3
5 6
1 0
1 3
4 2
69
2 × 2

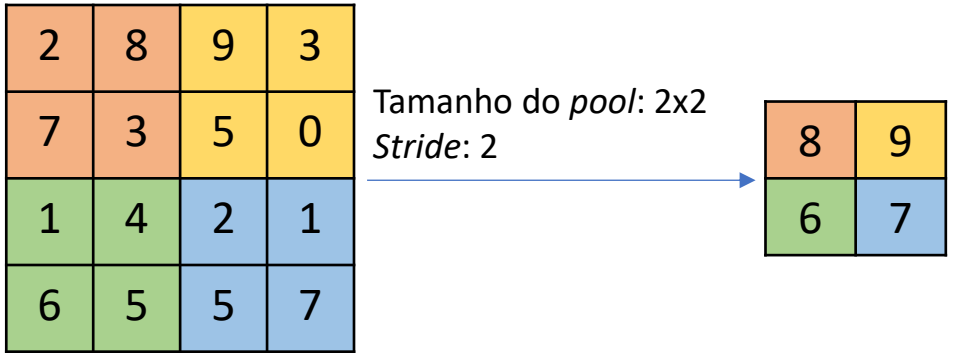

Input
Kernel
Input
Kernel
Output
0 1 2
3 4 5
6 7 8
3 × 3 × 3
0 1
2 3
2 × 2 × 3
0 1 2
3 4 5
6 7 8
0 1
2 3
3 1 5
0 2 7
8 4 9
5 6
1 0
9 2 3
3 1 4
8 0 5
1 3
4 2
69 85
2 × 2

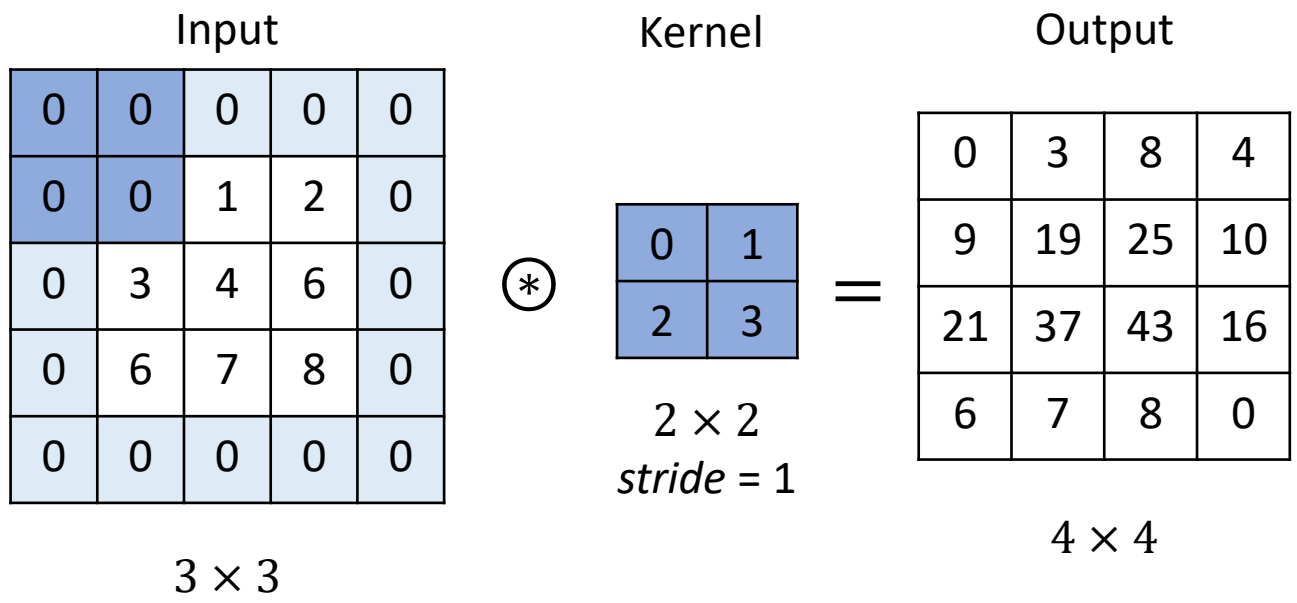

Input
Kernel
Input
Kernel
Output
0 1 2
3 4 5
6 7 8
3 × 3 × 3
0 1
2 3
2 × 2 × 3
3 1 5
0 2 7
8 4 9
9 2 3
3 1 4
8 0 5
0 1
2 3
5 6
1 0
1 3
4 2
69 85
95
2 × 2


Input
Kernel
Input
Kernel
Output
0 1 2
3 4 5
6 7 8
3 × 3 × 3
0 1
2 3
2 × 2 × 3
3 1 5
0 2 7
8 4 9
9 2 3
3 1 4
8 0 5
0 1
2 3
5 6
1 0
1 3
4 2
69 85
95 122
2 × 2

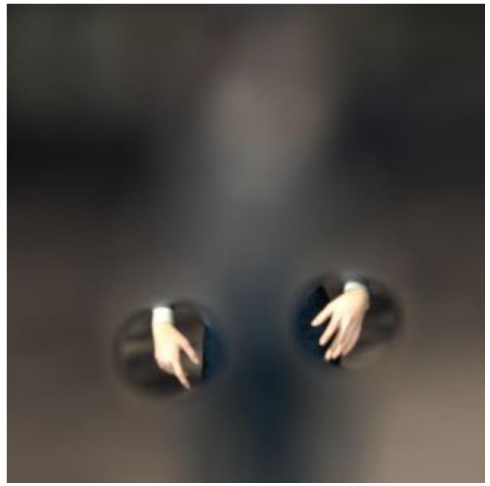

Input
Kernels
Output
Kernel, $W^{[1]}$
0 1 2
3 4 5
6 7 8
$3 \times 3 \times 3$
1 0
3 2
$2 \times 2 \times 3$
$f(W^{[1]} \circledast I + b_1)$
13 27
34 78
$2 \times 2 \times 2$
Kernel, $W^{[2]}$
2 3
5 4
$2 \times 2 \times 3$
$f(W^{[2]} \circledast I + b_2)$
Camada convolucional

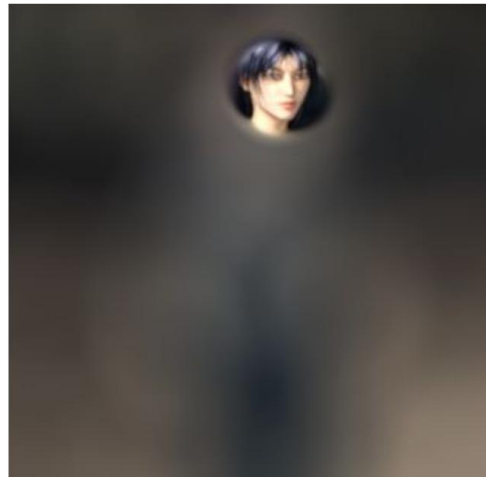
Max Pooling


2 8 9 3
7 3 5 0
1 4 2 1
6 5 5 7
Tamanho do *pool*: 2x2
Stride: 2
8 9
6 7

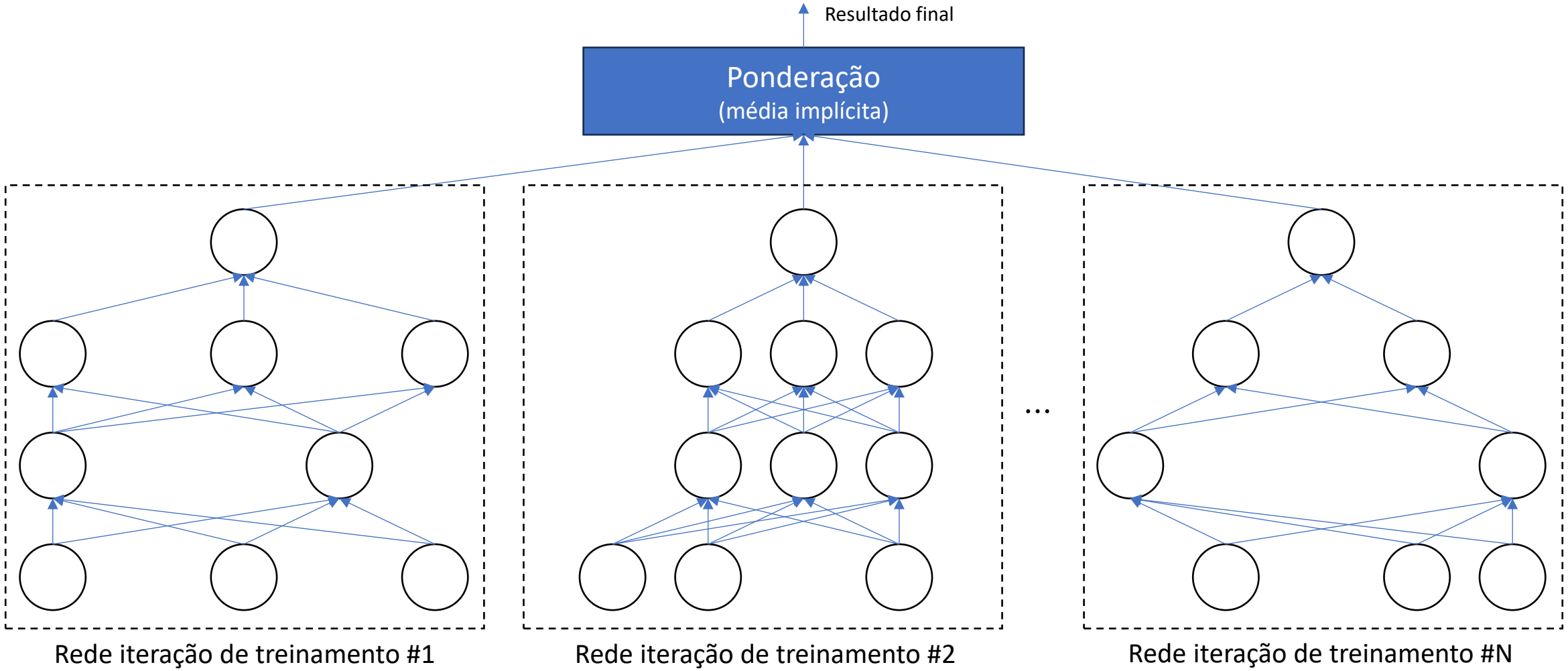

Input
Kernel
Output
0 0 0 0 0
0 0 1 2 0
0 3 4 6 0
0 6 7 8 0
0 0 0 0 0
3 × 3
⊛
0 1
2 3
2 × 2
stride = 1
=
0 3 8 4
9 19 25 10
21 37 43 16
6 7 8 0
4 × 4







Humano


Resultado final
Ponderação
(média implícita)
...
Rede iteração de treinamento #1
Rede iteração de treinamento #2
Rede iteração de treinamento #N